

PROGRAMACIÓN DE DISPOSITIVOS MÓVILES · ANDROID STUDIO · JAVA

# Recursos de Interfaz de Usuario

Guía de Actividades · Punto 3.7

---

Documento de estudio, desarrollo y sustentación

Proyecto integrador: **MiAgenda INTJEM**

Contenido: marco teórico · actividades resueltas · código completo  
guion de exposición · banco de preguntas de defensa

# Cómo usar este documento

Este material no está escrito para leerse una vez. Está escrito para **tres pasadas**, porque la evaluación no premia haber entregado código: premia poder explicarlo bajo preguntas cruzadas del docente.

| Pasada            | Qué hacer                                                                                                         | Tiempo | Resultado esperado                                                                     |
|-------------------|-------------------------------------------------------------------------------------------------------------------|--------|----------------------------------------------------------------------------------------|
| 1ª — Comprensión  | Leer el capítulo 2 (Fundamentos) entero, sin memorizar nada. Escribir en los márgenes lo que no se entienda.      | 60 min | Entender <i>por qué</i> existe el directorio <code>res/</code> , no solo qué contiene. |
| 2ª — Construcción | Teclear el código de los capítulos 5, 6 y 7 en Android Studio. <b>Teclear, no copiar y pegar.</b>                 | 4-5 h  | Proyecto funcionando + errores reales enfrentados.                                     |
| 3ª — Retención    | Estudiar solo el capítulo 9 (guion) y el 10 (banco de preguntas). Taparse las respuestas y responder en voz alta. | 90 min | Poder sustentar sin leer.                                                              |

**PRINCIPIO RECTOR DE TODO EL PUNTO 3.7**

Una aplicación Android bien construida **no escribe pantallas: declara recursos**. El código y el XML nunca contienen el valor; contienen una *referencia* al valor. El sistema operativo elige el valor concreto en tiempo de ejecución según el dispositivo, el idioma, la orientación y las preferencias del usuario.

Si el estudiante interioriza esa sola frase, todas las consignas de esta guía dejan de ser reglas arbitrarias y pasan a ser consecuencias lógicas.

## Advertencia previa sobre el nombre del proyecto

Las dos guías entregadas por la cátedra **no coinciden** en el nombre de la aplicación:

- La guía del punto **3.7** indica crear el proyecto `MiAgendaINTJEM` (“MiAgenda INTJEM”).
- La guía del punto **3.8** afirma ser continuación de `MiAgendaUPDS` (“MiAgenda UPDS”).

**ACCIÓN REQUERIDA ANTES DE CREAR EL PROYECTO**

Es un único proyecto que atraviesa las dos guías, de modo que **no puede tener dos nombres**. Hay que consultarlo con el docente antes de escribir la primera línea, porque renombrar el paquete de una aplicación Android a mitad de camino obliga a tocar el `applicationId`, el `namespace`, la estructura de carpetas `java/` y todos los `import` generados.

Este documento usa **MiAgendaINTJEM** por ser el nombre que aparece en la consigna de creación (AP-3.7.1). Si la cátedra confirma “UPDS”, basta con sustituir la cadena en el nombre del proyecto, en `@string/app_name` y en el nombre del tema; el resto del código no cambia.

## Mapa del documento

| Cap. | Contenido                                              | Para qué sirve                    |
|------|--------------------------------------------------------|-----------------------------------|
| 2    | Fundamentos conceptuales del sistema de recursos       | Base teórica de todo lo demás     |
| 3    | <b>AT-3.7.1</b> resuelta: mapa de recursos y tabla     | Entregable teórico 1              |
| 4    | <b>AT-3.7.2</b> resuelta: auditoría de malas prácticas | Entregable teórico 2              |
| 5    | <b>AP-3.7.1:</b> proyecto base, archivo por archivo    | Entregable práctico 1             |
| 6    | <b>AP-3.7.2:</b> calificadores de configuración        | Entregable práctico 2             |
| 7    | <b>AC-3.7.3:</b> sistema de diseño Material Design 3   | Entregable complementario (+10 %) |
| 8    | Errores frecuentes y cómo diagnosticarlos              | Laboratorio                       |
| 9    | Guion de exposición cronometrado                       | Sustentación                      |
| 10   | Banco de preguntas de defensa con respuestas           | Sustentación                      |
| 11   | Tarjetas de memorización y datos duros                 | Repaso final                      |
| 12   | Lista de verificación de entrega                       | Control                           |

# Fundamentos conceptuales

## El problema que resuelve el directorio `res/`

Una aplicación de escritorio tradicional puede permitirse escribir el texto, el color y el tamaño directamente dentro del código: se ejecuta en una pantalla conocida, en un idioma conocido, para un usuario cuya configuración no cambia.

Android es lo contrario. La **misma** aplicación, el **mismo** archivo `.apk`, debe funcionar simultáneamente en:

- un teléfono de 5" a 320 dpi y una tablet de 11" a 160 dpi;
- un dispositivo en español y otro en inglés, árabe o japonés;
- vertical y horizontal, cambiando de una a otra mientras el usuario la sostiene;
- modo claro y modo oscuro;
- un usuario con visión normal y otro que subió el tamaño de fuente del sistema al 200 %.

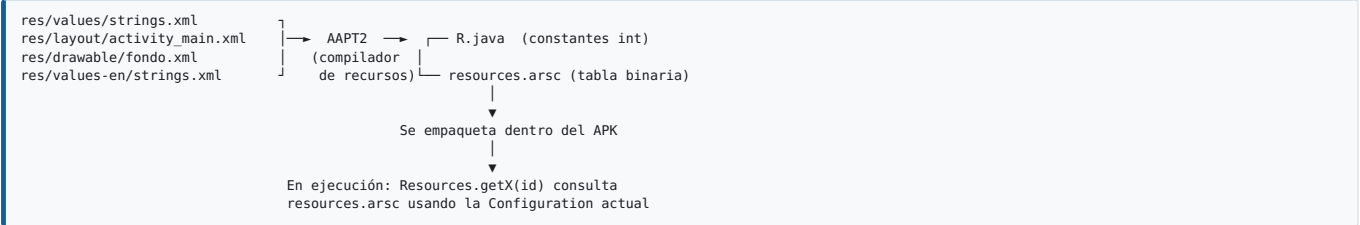
Un valor literal escrito en el layout es un valor **decidido por el programador en tiempo de compilación**. Un recurso es un valor **decidido por el sistema en tiempo de ejecución**. Esa es toda la diferencia, y es la razón por la que el punto 3.7 prohíbe los literales.

FORMULACIÓN PARA LA DEFENSA ORAL

“Externalizar un valor a `res/` convierte una decisión estática del programador en una decisión dinámica del sistema. El programador deja de decir *cuál* es el texto y pasa a decir *cuál es el rol* de ese texto; el sistema resuelve el valor concreto según la configuración del dispositivo.”

## Cómo se compila un recurso: de XML a la clase `R`

Entender esta cadena permite responder casi cualquier pregunta sobre recursos.



### Paso a paso:

1. **AAPT2** (*Android Asset Packaging Tool 2*) recorre `res/` y compila cada archivo a formato binario.
2. Genera la clase `R`, que contiene una constante entera por cada recurso. `R` es solo un diccionario de identificadores: no contiene valores, contiene números.
3. Genera `resources.arsc`, una tabla que asocia cada identificador con *todas* sus variantes (español, inglés, modo noche, horizontal...).
4. En ejecución, cuando se pide `getString(R.string.app_name)`, el framework toma el entero, busca la fila en `resources.arsc` y devuelve **la variante que mejor coincide con la `Configuration` actual** del dispositivo.

### Anatomía de un identificador de recurso

```
R.string.app_name == 0x7F0A0001
├── 0x7F = la propia aplicación
├── 0x0A = android (recursos del sistema)
└── 0001 = entry: número del recurso dentro de su tipo
    type: tipo de recurso (string, color, layout...)
    package: 0x7F = la propia aplicación
    0x01 = android (recursos del sistema)
```

PREGUNTA TÍPICA DEL DOCENTE

“¿Qué es realmente `R.string.app_name` ?”

Respuesta correcta: es un `public static final int` generado automáticamente por AAPT2. **No es el texto**. Es un identificador numérico que el framework usa como clave de búsqueda en la tabla `resources.arsc`. Por eso `textView.setText(R.string.saludo)` muestra el saludo pero `textView.setText(String.valueOf(R.string.saludo))` mostraría un número.

## Anatomía completa del directorio `res/`

|                                |                                                                 |
|--------------------------------|-----------------------------------------------------------------|
| <code>app/src/main/res/</code> |                                                                 |
| ├─ drawable/                   | Gráficos: vectores XML, shapes, selectors, PNG                  |
| ├─ drawable-hdpi/              | Variantes por densidad de pantalla (mapas de bits)              |
| ├─ mipmap-mdpi/ ...            | EXCLUSIVAMENTE el ícono de la aplicación                        |
| ├─ layout/                     | Estructura de pantallas en XML                                  |
| ├─ layout-land/                | Variante horizontal de esas mismas pantallas                    |
| ├─ values/                     |                                                                 |
| │ └─ strings.xml               | Cadenas de texto, string-array, plurals                         |
| │ └─ colors.xml                | Colores                                                         |
| │ └─ dimens.xml                | Dimensiones (dp, sp)                                            |
| │ └─ themes.xml                | Temas y estilos                                                 |
| │ └─ attrs.xml                 | Atributos personalizados                                        |
| ├─ values-en/                  | Mismos archivos, variante en inglés                             |
| ├─ values-night/               | Mismos archivos, variante para modo oscuro                      |
| ├─ values-sw600dp/             | Mismos archivos, variante para tablet                           |
| ├─ menu/                       | Menús de Toolbar, contextuales, de navegación                   |
| ├─ anim/ y animator/           | Animaciones de vista y de propiedad                             |
| ├─ xml/                        | Configuraciones arbitrarias (preferencias, backup)              |
| ├─ raw/                        | Archivos crudos (audio, JSON) accesibles por <code>R.raw</code> |
| └─ font/                       | Fuentes tipográficas y familias de fuentes                      |

### Tabla de referencia por subdirectorio

| Carpeta            | Qué aloja            | Ejemplo de contenido                       | Acceso XML                  | Acceso Java                                                   |
|--------------------|----------------------|--------------------------------------------|-----------------------------|---------------------------------------------------------------|
| values/strings.xml | Cadenas              | <string name="app_name">MiAgenda</string>  | @string/app_name            | getString(R.string.app_name)                                  |
| values/colors.xml  | Colores              | <color name="primario">#0B5FA5</color>     | @color/primario             | ContextCompat.getColor(this, R.color.primario)                |
| values/dimens.xml  | Dimensiones          | <dimen name="espaciado_medio">16dp</dimen> | @dimen/espaciado_medio      | getResources().getDimensionPixelSize(R.dimen.espaciado_medio) |
| values/themes.xml  | Estilos y temas      | <style name="EstiloBotonPrincipal">        | @style/EstiloBotonPrincipal | setTheme(R.style.Theme_MiAgendaINTJEM)                        |
| drawable/          | Gráficos             | shape, vector, selector, PNG               | @drawable/fondo_boton       | ContextCompat.getDrawable(this, R.drawable.fondo_boton)       |
| mipmap-*/          | Ícono de la app      | ic_launcher.png                            | @mipmap/ic_launcher         | R.mipmap.ic_launcher                                          |
| layout/            | Pantallas            | activity_main.xml                          | @layout/activity_main       | setContentView(R.layout.activity_main)                        |
| menu/              | Menús                | menu_principal.xml                         | @menu/menu_principal        | getMenuInflater().inflate(R.menu.menu_principal, menu)        |
| anim/              | Animaciones de vista | fade_in.xml                                | @anim/fade_in               | AnimationUtils.loadAnimation(this, R.anim.fade_in)            |
| raw/               | Archivos crudos      | tono.mp3                                   | @raw/tono                   | getResources().openRawResource(R.raw.tono)                    |

**POR QUÉ MIPMAP EXISTE SEPARADO DE DRAWABLE**

Ésta es la pregunta que el criterio de logro de AT-3.7.1 exige responder, y la que más se contesta mal.

**Razón técnica:** cuando se generan APK o bundles separados por densidad, el sistema de construcción **elimina** las carpetas *drawable-dpi* que no corresponden a la densidad del dispositivo, para reducir el tamaño de descarga. Las carpetas *mipmap-* **se conservan todas**.

**Por qué hace falta conservarlas:** el lanzador (launcher) del dispositivo puede pedir el ícono en una densidad *superior* a la de la pantalla. Un launcher en un teléfono xhdpi puede solicitar la versión xxhdpi del ícono para dibujarlo ampliado sin pixelarlo. Si esa carpeta hubiera sido eliminada, el ícono se vería borroso.

**Regla práctica:** *mipmap* es solo para *ic\_launcher*. Cualquier otro gráfico va en *drawable*. Meter imágenes de la interfaz en *mipmap* es un error conceptual que además engorda el APK.

**Sintaxis de referencia: @ frente a ?**

| Sintaxis             | Significado                                     | Resuelto en                                 | Ejemplo                                            |
|----------------------|-------------------------------------------------|---------------------------------------------|----------------------------------------------------|
| @tipo/nombre         | Referencia a un <b>recurso concreto</b>         | Tiempo de compilación / búsqueda directa    | @color/primario                                    |
| @android:tipo/nombre | Recurso del <b>sistema operativo</b>            | Igual, pero en el paquete android           | @android:color/transparent                         |
| ?attr/nombre         | Referencia a un <b>atributo del tema activo</b> | Tiempo de ejecución, según el tema aplicado | ?attr/colorPrimary                                 |
| @+id/nombre          | <b>Crea</b> un nuevo identificador              | Compilación                                 | android:id="@+id/btnIngresar"                      |
| @id/nombre           | <b>Usa</b> un identificador ya creado           | Compilación                                 | app:layout_constraintTop_toBottomOf="@id/tvTitulo" |

**LA DISTINCIÓN QUE SEPARA UN 7 DE UN 10 EN LA SUSTENTACIÓN**

@color/md\_primary dice “este color exacto”. ?attr/colorPrimary dice “el color que el tema activo haya designado como primario”.

El primero es rígido: si cambia el tema, el color no cambia. El segundo es el mecanismo que permite que el modo oscuro funcione **sin una sola línea de Java**. Material Design 3 se apoya casi exclusivamente en ?attr/.

**Unidades: px, dp y sp**

**Definiciones**

- px (pixel):** un punto físico de la pantalla. Su tamaño real depende de la densidad del dispositivo. **Nunca se usa en la interfaz.**
- dp o dip (density-independent pixel):** unidad abstracta anclada a una pantalla de referencia de 160 dpi. 1 dp mide siempre aproximadamente lo mismo físicamente, sin importar el dispositivo.
- sp (scale-independent pixel):** igual que dp, pero además se multiplica por el factor de escala de fuente que el usuario configuró en Ajustes → Pantalla → Tamaño de fuente. **Se usa únicamente para tamaños de texto.**

**Las dos fórmulas que conviene memorizar**

|    |   |    |   |                                                    |
|----|---|----|---|----------------------------------------------------|
| px | = | dp | × | (densidad_de_pantalla_dpi / 160)                   |
| px | = | sp | × | (densidad_de_pantalla_dpi / 160) × fontScale       |
|    |   |    |   | └─ preferencia del usuario (1.0 normal, hasta 2.0) |

**Tabla de densidades (bucket, factor y traducción)**

| Bucket                   | dpi aproximado | Factor      | 1 dp equivale a | 16 dp equivalen a |
|--------------------------|----------------|-------------|-----------------|-------------------|
| ldpi                     | 120            | 0.75x       | 0.75 px         | 12 px             |
| <b>mdpi</b> (referencia) | <b>160</b>     | <b>1.0x</b> | <b>1 px</b>     | <b>16 px</b>      |
| hdpi                     | 240            | 1.5x        | 1.5 px          | 24 px             |
| xhdpi                    | 320            | 2.0x        | 2 px            | 32 px             |
| xxhdpi                   | 480            | 3.0x        | 3 px            | 48 px             |
| xxxhdpi                  | 640            | 4.0x        | 4 px            | 64 px             |

**LA CONSECUENCIA CONCRETA DE USAR PX**

Un botón declarado como `android:layout_width="320px"` mide:

- en un dispositivo mdpi: 320 px = **320 dp** → ocupa casi toda la pantalla;
- en un dispositivo xxhdpi: 320 px = **106,7 dp** → ocupa un tercio y se ve diminuto.

El mismo número produce dos interfaces completamente distintas. Ése es el argumento, y no “porque la documentación lo dice”.

## Calificadores de configuración y algoritmo de selección

Un calificador es un sufijo añadido al nombre de la carpeta: `values-en`, `layout-land`, `values-night`, `values-sw600dp`. Crea una **variante del mismo recurso**, nunca un recurso nuevo.

**Orden de precedencia (fragmento relevante, de mayor a menor)**

| #  | Calificador                         | Ejemplo                                    | Usado en esta guía    |
|----|-------------------------------------|--------------------------------------------|-----------------------|
| 1  | Código de país móvil (MCC/MNC)      | <code>mcc736</code>                        | —                     |
| 2  | <b>Idioma y región</b>              | <code>en</code> , <code>es-rB0</code>      | Sí (AP-3.7.2 punto 2) |
| 3  | Dirección de layout                 | <code>ldrtl</code>                         | —                     |
| 4  | <b>Ancho mínimo (smallestWidth)</b> | <code>sw600dp</code>                       | Sí (AP-3.7.2 punto 5) |
| 5  | Ancho disponible                    | <code>w720dp</code>                        | —                     |
| 6  | Alto disponible                     | <code>h720dp</code>                        | —                     |
| 7  | Tamaño de pantalla                  | <code>large</code>                         | — (obsoleto)          |
| 8  | <b>Orientación</b>                  | <code>port</code> , <code>land</code>      | Sí (AP-3.7.2 punto 1) |
| 9  | Modo de interfaz                    | <code>car</code> , <code>television</code> | —                     |
| 10 | <b>Modo noche</b>                   | <code>night</code> , <code>notnight</code> | Sí (AP-3.7.2 punto 3) |
| 11 | <b>Densidad de pantalla</b>         | <code>hdpi</code> , <code>xxhdpi</code>    | Sí (AP-3.7.2 punto 4) |
| 12 | Versión de plataforma               | <code>v24</code> , <code>v31</code>        | Ocasional             |

**Algoritmo de selección del recurso**

1. **Descarta** todas las carpetas cuyo calificador **contradiga** la configuración actual. Un dispositivo en vertical descarta `layout-land` de inmediato.
2. Toma el calificador de **mayor precedencia** que aún no se haya evaluado.
3. ¿Alguna carpeta candidata lo incluye? Si no, vuelve al paso 2 con el siguiente calificador.
4. Si sí, **elimina** todas las carpetas que **no** lo incluyan.
5. Repite hasta que quede una sola carpeta.

**CONSECUENCIA QUE HAY QUE SABER EXPLICAR**

Como `sw600dp` (precedencia 4) está **por encima** de `land` (precedencia 8), en una tablet en horizontal el sistema elegirá `values-sw600dp` antes que cualquier variante de orientación. El orden no es arbitrario: va de lo más estable (el hardware, el idioma) a lo más volátil (la orientación, que cambia cada vez que el usuario gira el aparato).

**La densidad es la única excepción al algoritmo:** nunca se descarta por completo. Si no existe la densidad exacta, el sistema toma la más cercana y **escala** el mapa de bits. Por eso un drawable vectorial —que no se escala, se redibuja— es siempre superior.

## ConstraintLayout: lo mínimo indispensable

`ConstraintLayout` construye la interfaz declarando **relaciones entre vistas**, no anidando contenedores. Reemplaza jerarquías profundas de `LinearLayout` por una jerarquía plana, que el sistema mide y dibuja en menos pasadas.

**Reglas de oro**

1. **Toda vista necesita al menos una restricción horizontal y una vertical.** Sin ellas se dibuja en la esquina superior izquierda al ejecutar (aunque el editor la muestre bien).
2. `0dp` **no significa cero.** Significa `MATCH_CONSTRAINT`: “ocupa todo el espacio que definan mis restricciones”. Es el equivalente correcto de `match_parent` dentro de un `ConstraintLayout`.
3. **Nunca fijar anchos absolutos** en la dimensión que debe adaptarse.
4. `bias` reparte el espacio sobrante entre dos restricciones opuestas: `0.0` pega a la izquierda/arriba, `0.5` centra, `1.0` pega a la derecha/abajo.
5. `Guideline` es una línea invisible de referencia, posicionada en porcentaje (`layout_constraintGuide_percent`) o en dp. No ocupa espacio y es la herramienta correcta para dividir la pantalla en el layout horizontal.
6. **Cadenas (chains):** dos o más vistas restringidas entre sí en el mismo eje. Modos: `spread`, `spread_inside`, `packed`. Es la manera correcta de colocar dos botones lado a lado con reparto proporcional.

| Atributo                                                         | Para qué sirve                                                 |
|------------------------------------------------------------------|----------------------------------------------------------------|
| <code>app:layout_constraintTop_toBottomOf="@id/x"</code>         | Mi borde superior se ancla al borde inferior de <code>x</code> |
| <code>app:layout_constraintStart_toStartOf="parent"</code>       | Mi borde inicial se alinea con el del contenedor               |
| <code>app:layout_constraintHorizontal_bias="0.5"</code>          | Reparto del espacio sobrante en el eje horizontal              |
| <code>app:layout_constraintHorizontal_chainStyle="spread"</code> | Estilo de la cadena horizontal                                 |
| <code>app:layout_constraintDimensionRatio="16:9"</code>          | Relación de aspecto fija                                       |

**SOBRE START/END FRENTE A LEFT/RIGHT**

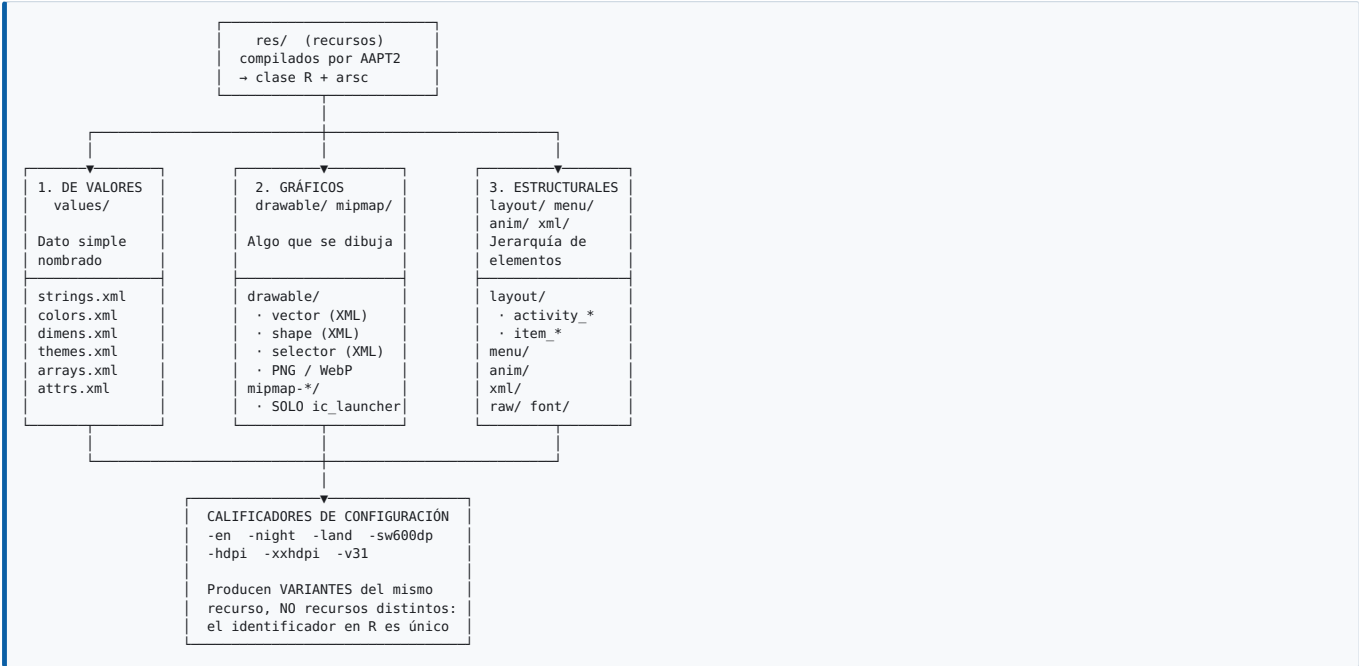
Usar siempre `Start` y `End`, nunca `Left` y `Right`. `Start` es la izquierda en español e inglés, pero es la **derecha** en árabe y hebreo. Es el mismo principio de externalización aplicado a la geometría: se declara el rol ("el borde por donde empieza la lectura"), no la posición física.

# AT-3.7.1 · Mapa de recursos del proyecto

**Modalidad:** teórica · 60 min · en pareja  
**Producto exigido:** mapa conceptual en una sola hoja + tabla con al menos ocho recursos.

## Esquema jerárquico propuesto

El siguiente esquema está pensado para transcribirse a Draw.io o Lucidchart. La estructura de tres ramas es deliberada: es exactamente la clasificación que el criterio de logro exige distinguir.



## Tabla complementaria de recursos (12 entradas)

La consigna pide ocho; se entregan doce para tener margen ante preguntas.

| #  | Recurso                 | Carpeta            | Acceso desde XML                 | Acceso desde Java                                                  |
|----|-------------------------|--------------------|----------------------------------|--------------------------------------------------------------------|
| 1  | app_name                | values/strings.xml | @string/app_name                 | getString(R.string.app_name)                                       |
| 2  | mensaje_bienvenida      | values/strings.xml | @string/mensaje_bienvenida       | getString(R.string.mensaje_bienvenida)                             |
| 3  | primario                | values/colors.xml  | @color/primario                  | ContextCompat.getColor(this, R.color.primario)                     |
| 4  | texto_sobre_primario    | values/colors.xml  | @color/texto_sobre_primario      | ContextCompat.getColor(this, R.color.texto_sobre_primario)         |
| 5  | espaciado_medio         | values/dimens.xml  | @dimen/espaciado_medio           | getResources().getDimensionPixelSize(R.dimen.espaciado_medio)      |
| 6  | texto_titulo            | values/dimens.xml  | @dimen/texto_titulo              | getResources().getDimension(R.dimen.texto_titulo)                  |
| 7  | EstiloBotonPrincipal    | values/themes.xml  | @style/EstiloBotonPrincipal      | new ContextThemeWrapper(this, R.style.EstiloBotonPrincipal)        |
| 8  | fondo_boton_redondeado  | drawable/          | @drawable/fondo_boton_redondeado | ContextCompat.getDrawable(this, R.drawable.fondo_boton_redondeado) |
| 9  | ic_agenda               | drawable/ (vector) | @drawable/ic_agenda              | imageView.setImageResource(R.drawable.ic_agenda)                   |
| 10 | ic_launcher             | mipmap-*/          | @mipmap/ic_launcher              | R.mipmap.ic_launcher                                               |
| 11 | activity_main           | layout/            | @layout/activity_main            | setContentView(R.layout.activity_main)                             |
| 12 | materias (string-array) | values/arrays.xml  | @array/materias                  | getResources().getStringArray(R.array.materias)                    |

**PRECISIÓN QUE CONVIENE AÑADIR EN EL INFORME**

Las tres variantes de lectura de dimensiones en Java **no son equivalentes**, y preguntarlo es una forma rápida de separar a quien entendió de quien copió:

- `getDimension(id)` → devuelve `float` en píxeles, con decimales.
- `getDimensionPixelSize(id)` → devuelve `int`, **redondeando**; garantiza al menos 1 px si el valor es mayor que cero. Es el correcto para anchos y altos.
- `getDimensionPixelOffset(id)` → devuelve `int`, **truncando**. Es el correcto para desplazamientos.

Las tres ya devuelven píxeles: la conversión de dp a px la hace el framework, no el programador.

## Justificación escrita del criterio de logro

Texto sugerido para el informe, listo para adaptar:

El directorio `res/` se organiza en tres familias funcionales. Los **recursos de valores** (`values/`) almacenan datos simples nombrados —cadenas, colores, dimensiones, estilos— que se compilan en la tabla `resources.arsc` y se identifican por un entero de la clase `R`. Los **recursos gráficos** (`drawable/`, `mipmap/`) almacenan todo aquello que se dibuja, ya sea de forma declarativa en XML (`shape`, `vector`, `selector`) o como mapa de bits. Los **recursos estructurales** (`layout/`, `menu/`, `anim/`) describen jerarquías de elementos que el framework infla en objetos en tiempo de ejecución.

La distinción entre `drawable/` y `mipmap/` no es estilística sino de empaquetado: al generar distribuciones segmentadas por densidad, el sistema de construcción elimina las carpetas `drawable-*dpi` ajenas a la densidad del dispositivo, pero conserva íntegramente las carpetas `mipmap-*`. Esto es necesario porque el lanzador del dispositivo puede solicitar el ícono de la aplicación en una densidad superior a la de la pantalla para dibujarlo ampliado sin pérdida de nitidez. Por esa razón `mipmap/` se reserva exclusivamente al ícono de la aplicación.

# AT-3.7.2 · Auditoría de malas prácticas

Modalidad: teórica · 45 min · en pareja  
Producto exigido: tabla de auditoría + respuestas a tres preguntas de discusión.

## Fragmento entregado por la cátedra

```
<TextView
  android:layout_width="320px"
  android:layout_height="wrap_content"
  android:text="Bienvenido al sistema"
  android:textSize="18px"
  android:textColor="#FF5722"
  android:padding="16px" />
```

## Tabla de auditoría

| # | Línea observada                      | Tipo de falla                                  | Consecuencia sobre la aplicación                                                                                                                     | Línea corregida                                                   |
|---|--------------------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| 1 | layout_width="320px"                 | Unidad incorrecta + dimensión no parametrizada | En mdpi ocupa 320 dp (casi toda la pantalla); en xxhdpi solo 106,7 dp. La misma interfaz se ve distinta en cada dispositivo.                         | android:layout_width="0dp" con restricciones, o "wrap_content"    |
| 2 | layout_width fijo                    | Ancho absoluto                                 | Se desborda o deja huecos al rotar y en tablets. No participa del sistema de restricciones.                                                          | 0dp + constraintStart/End_toStartOf/EndOf="parent"                |
| 3 | android:text="Bienvenido al sistema" | Literal embebido                               | Imposible de traducir sin recompilar. values-en no tiene punto de intervención. Lint lo marca como HardcodedText.                                    | android:text="@string/mensaje_bienvenida"                         |
| 4 | android:textSize="18px"              | Unidad incorrecta para texto                   | Ignora el factor de escala de fuente del sistema. Un usuario con baja visión que sube la fuente al 200 % sigue viendo el texto igual. Lint: SpUsage. | android:textSize="@dimen/texto_cuerpo" con valor en sp            |
| 5 | android:textColor="#FF5722"          | Color no externalizado                         | No puede variar en values-night; queda idéntico en modo oscuro. Además su contraste sobre blanco es 3,16:1, por debajo del mínimo 4,5:1 de WCAG.     | android:textColor="@color/texto_principal" o ?attr/colorOnSurface |
| 6 | android:padding="16px"               | Unidad incorrecta + no parametrizada           | El espaciado se encoge proporcionalmente en pantallas densas; los objetivos táctiles quedan por debajo de los 48 dp recomendados.                    | android:padding="@dimen/espaciado_medio"                          |
| 7 | Ausencia de android:id               | Falta estructural                              | No puede referenciarse desde Java ni servir de ancla a otras vistas; el estado no se guarda automáticamente al rotar.                                | android:id="@+id/tvBienvenida"                                    |

## Fragmento corregido

```
<TextView
  android:id="@+id/tvBienvenida"
  android:layout_width="0dp"
  android:layout_height="wrap_content"
  android:text="@string/mensaje_bienvenida"
  android:textSize="@dimen/texto_cuerpo"
  android:textColor="@color/texto_principal"
  android:padding="@dimen/espaciado_medio"
  app:layout_constraintTop_toBottomOf="@id/ivLogo"
  app:layout_constraintStart_toStartOf="parent"
  app:layout_constraintEnd_toEndOf="parent" />
```

Con los recursos correspondientes declarados aparte:

```
<!-- values/strings.xml -->
<string name="mensaje_bienvenida">Bienvenido al sistema</string>

<!-- values/dimens.xml -->
<dimen name="texto_cuerpo">16sp</dimen>
<dimen name="espaciado_medio">16dp</dimen>

<!-- values/colors.xml -->
<color name="texto_principal">#1A1C1E</color>
```

**DATO VERIFICABLE QUE IMPRESIONA EN LA SUSTENTACIÓN**  
El color #FF5722 del fragmento original tiene sobre fondo blanco una razón de contraste de 3,16:1. WCAG 2.1 nivel AA exige 4,5:1 para texto normal. El fragmento entregado por la cátedra **no es solo poco mantenible: es inaccesible**. Decir esto con el número exacto demuestra que se verificó y no se supuso.

## Preguntas de discusión resueltas

### 1. ¿Qué ocurre con el texto si el usuario aumenta el tamaño de fuente del sistema y el desarrollador usó px en lugar de sp?

**Respuesta breve:** no ocurre nada. Y ése es exactamente el problema.

**Respuesta desarrollada.** El sistema aplica la preferencia de tamaño de fuente mediante el campo `fontScale` del objeto `Configuration`. Ese factor interviene únicamente en la conversión de `sp` a píxeles:

```
px_finales = sp × (dpi / 160) × fontScale
px_finales = px          ← fontScale no aparece
```

Al declarar `18px`, el desarrollador ha fijado el resultado final de la conversión y ha eliminado el punto donde el sistema podía intervenir. El texto queda congelado a 18 píxeles físicos.

Consecuencias en cadena:

1. **Accesibilidad rota.** Un usuario con baja visión que configura la fuente al 200 % no percibe ningún cambio. Se incumple el criterio WCAG 2.1 §1.4.4 (*Resize Text*).
2. **Inconsistencia visual.** Si parte de la interfaz usa `sp` y parte usa `px`, al subir la fuente unos textos crecen y otros no: la pantalla se descuadra y los textos se solapan.
3. **Además, tamaño físico variable.** Independientemente del `fontScale`, 18 px en mdpi son 18 dp legibles, pero en xxhdpi son 6 dp: prácticamente ilegibles.
4. **Desde Android 14** el escalado de fuentes es *no lineal*: los textos ya grandes crecen proporcionalmente menos que los pequeños, para que las pantallas no se desarmen. Ese refinamiento tampoco alcanza al texto declarado en `px`.

**Regla que se deriva:** `sp` solo para tamaños de texto; `dp` para todo lo demás; `px` nunca en la interfaz. La única excepción legítima al uso de `px` es la manipulación de mapas de bits a nivel de píxel.

### 2. ¿Por qué la externalización de cadenas es requisito previo a cualquier internacionalización?

**Respuesta breve:** porque la internacionalización no traduce texto: **selecciona recursos**. Si el texto no es un recurso, no hay nada que seleccionar.

**Respuesta desarrollada.** El mecanismo de internacionalización de Android funciona así:

1. El dispositivo tiene un `Locale` activo (`es_B0`, `en_US`, ...) dentro de su `Configuration`.
2. El desarrollador crea carpetas `values-en/`, `values-pt/`, con el mismo `strings.xml` y los mismos nombres de recurso.
3. En ejecución, `getString(R.string.mensaje_bienvenida)` resuelve el identificador contra `resources.arsc` y devuelve la fila correspondiente al `Locale` activo.
4. Si no existe variante para ese idioma, se usa `values/` como **respaldo por omisión**. De ahí que `values/` deba contener siempre el idioma base completo.

Un literal escrito en el layout se compila **dentro** del archivo binario del layout. No tiene identificador, no tiene fila en la tabla, no tiene punto de bifurcación. La única manera de traducirlo sería duplicar el layout completo en `layout-en/`, lo cual multiplica el mantenimiento y vuelve a introducir el problema en cada cambio de diseño.

Beneficios adicionales que la externalización habilita y conviene mencionar:

- **Plurales correctos** con `<plurals>`, que resuelven que el español tenga dos formas y el árabe seis.
- **Parámetros posicionales** (`%1$s`, `%2$d`), imprescindibles porque el orden de las palabras cambia entre idiomas.
- **Flujo de trabajo profesional:** el traductor recibe un XML y nunca toca el código.
- **Soporte RTL:** `android:supportsRtl="true"` más `Start/End` permiten que la interfaz se espeje en árabe sin rehacer los layouts.

### 3. ¿Qué ventaja operativa concreta obtiene un equipo al centralizar la paleta en colors.xml?

Cinco ventajas, ordenadas de la más obvia a la más importante:

| Ventaja                                   | Explicación operativa                                                                                                                                                                                                                                            |
|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Punto único de cambio                     | Un cambio de identidad institucional se resuelve editando un archivo, no rastreando decenas de literales dispersos con similares pero no idénticos ( <code>#0B5FA5</code> , <code>#0B5FA6</code> , <code>#0C5FA5</code> ).                                       |
| Habilita <code>values-night</code> gratis | El modo oscuro consiste en redefinir los mismos nombres con otros valores. Si no hay nombres, no hay nada que redefinir y el modo oscuro exige lógica en Java.                                                                                                   |
| Auditoría de accesibilidad posible        | Se puede verificar el contraste de toda la aplicación revisando un archivo de veinte líneas, en vez de rastrear el proyecto entero.                                                                                                                              |
| Nomenclatura semántica                    | Nombrar <code>texto_sobre_primario</code> en vez de <code>blanco</code> permite que el valor cambie sin que el nombre mienta. Un recurso llamado <code>blanco</code> que en modo oscuro vale <code>#1A1C1E</code> es una trampa para el siguiente desarrollador. |
| Contrato compartido dentro del equipo     | El diseñador entrega roles, no códigos hexadecimales. Los dos integrantes de la pareja usan el mismo vocabulario y los conflictos de merge en Git se reducen a un archivo pequeño y ordenado.                                                                    |

#### FRASE DE CIERRE PARA ESTA PREGUNTA

“Centralizar la paleta convierte el color de un *detalle de implementación repetido* en una *decisión de diseño declarada una sola vez*. El beneficio no es estético: es que el costo de cambiar deja de crecer con el tamaño del proyecto.”

# AP-3.7.1 · Proyecto base “MiAgenda INTJEM”

**Modalidad:** práctica · 90 min · en pareja  
**Regla absoluta de esta actividad:** ningún valor literal en los archivos de layout.

## Paso 0 · Creación y configuración del proyecto

| Parámetro                    | Valor                                                                 |
|------------------------------|-----------------------------------------------------------------------|
| Plantilla                    | <b>Empty Views Activity</b> (no “Empty Activity”, que genera Compose) |
| Name                         | MiAgendaINTJEM                                                        |
| Package name                 | com.example.miagendaintjem                                            |
| Language                     | <b>Java</b>                                                           |
| Minimum SDK                  | <b>API 24 (Android 7.0 Nougat)</b>                                    |
| Build configuration language | Groovy DSL o Kotlin DSL, indistinto                                   |

**ERROR DE ARRANQUE MÁS FRECUENTE**

En las versiones recientes de Android Studio, la plantilla “**Empty Activity**” genera un proyecto en **Jetpack Compose y Kotlin**, sin carpeta `res/layout` y sin XML. Toda esta guía sería inaplicable. Hay que elegir “**Empty Views Activity**”, que es la plantilla basada en el sistema de vistas clásico.

### app/build.gradle — bloques relevantes

```
android {
    namespace 'com.example.miagendaintjem'
    compileSdk 34

    defaultConfig {
        applicationId "com.example.miagendaintjem"
        minSdk 24
        targetSdk 34
        versionCode 1
        versionName "1.0"
    }

    compileOptions {
        sourceCompatibility JavaVersion.VERSION_1_8
        targetCompatibility JavaVersion.VERSION_1_8
    }
}

dependencies {
    implementation 'androidx.appcompat:appcompat:1.7.0'
    implementation 'com.google.android.material:material:1.12.0'
    implementation 'androidx.constraintlayout:constraintlayout:2.1.4'
    implementation 'androidx.core:core:1.13.1'
}
```

`sourceCompatibility 1.8` es necesario para poder usar expresiones lambda ( `v -> ...` ) en los escuchadores de clic. Sin esa línea, el compilador rechaza esa sintaxis.

## Paso 1 · res/values/colors.xml

Consigna: al menos cinco colores con nombres semánticos. Se entregan siete.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <!-- Identidad institucional -->
    <color name="primario">#0B5FA5</color>
    <color name="primario_oscuro">#07406F</color>
    <color name="acento">#00897B</color>

    <!-- Superficies y texto -->
    <color name="superficie">#FDFCFE</color>
    <color name="texto_principal">#1A1C1E</color>
    <color name="texto_sobre_primario">#FFFFFF</color>

    <!-- Estado -->
    <color name="error">#BA1A1A</color>
</resources>
```

**QUÉ SIGNIFICA “NOMBRE SEMÁNTICO” (Y QUÉ NO)**

Semántico significa nombrado **por su función**, no por su apariencia.

**Incorrecto:** azul, azul\_claro, naranja, blanco. El nombre describe el valor. Cuando llegue el modo oscuro y blanco tenga que valer #1A1C1E, el nombre pasará a mentir.

**Correcto:** primario, superficie, texto\_sobre\_primario, error. El nombre describe el rol. El valor puede cambiar entre values/ y values-night/ sin que el nombre deje de ser cierto.

*Nota:* la tabla de ejemplo de la propia guía usa azul\_primario, que mezcla las dos convenciones. Conviene señalarlo en el informe y adoptar la forma puramente semántica, que es además la que exige Material 3 (ver capítulo 7).

## Paso 2 · res/values/dimens.xml

Consigna: al menos cuatro dimensiones reutilizables. Se entregan once, agrupadas por propósito.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <!-- Escala de espaciado: múltiplos de 8dp (retícula de Material Design) -->
  <dimen name="espaciado_pequeno">8dp</dimen>
  <dimen name="espaciado_medio">16dp</dimen>
  <dimen name="espaciado_grande">24dp</dimen>

  <!-- Forma y elevación -->
  <dimen name="radio_esquina">12dp</dimen>
  <dimen name="borde_boton">1dp</dimen>
  <dimen name="elevacion_boton">2dp</dimen>

  <!-- Componentes -->
  <dimen name="altura_boton">48dp</dimen>
  <dimen name="tamano_logo">120dp</dimen>

  <!-- Tipografía: SIEMPRE en sp -->
  <dimen name="texto_titulo">24sp</dimen>
  <dimen name="texto_cuerpo">16sp</dimen>
  <dimen name="texto_boton">16sp</dimen>
</resources>
```

## DOS NÚMEROS QUE CONVIENE PODER JUSTIFICAR

**8 dp** — Material Design organiza el espaciado en una retícula de 8 dp. Todos los márgenes y rellenos son múltiplos de 8. Esto no es capricho estético: una escala restringida elimina discusiones y produce ritmo visual coherente sin esfuerzo.

**48 dp** — es el tamaño mínimo recomendado de un objetivo táctil, porque corresponde aproximadamente a 9 mm, que es el área media de contacto de la yema de un dedo adulto. Un botón menor a 48 dp produce errores de pulsación medibles. Declararlo como `@dimen/altura_boton` garantiza que ninguna pantalla lo viole por descuido.

## Paso 3 · `res/values/strings.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <string name="app_name">MiAgenda INTJEM</string>

  <!-- Pantalla principal -->
  <string name="titulo_encabezado">Agenda Académica</string>
  <string name="mensaje_bienvenida">Bienvenido al sistema. Organiza tus tareas, materias y fechas de entrega en un solo lugar.</string>
  <string name="boton_ingresar">Ingresar</string>
  <string name="boton_acerca_de">Acerca de</string>

  <!-- Accesibilidad -->
  <string name="desc_logo">Ícono de la agenda académica</string>

  <!-- Mensajes -->
  <string name="msg_ingresar">Abriendo la agenda...</string>
  <string name="msg_acerca_de">%1$s · versión 1.0 · Punto 3.7</string>
</resources>
```

`%1$s` es un **parámetro posicional**. El `1` indica que es el primer argumento y el `$s` que es una cadena. Se usa numeración posicional en lugar de un simple `%s` porque en otros idiomas el orden de las palabras cambia y el traductor debe poder reordenar los parámetros sin tocar el código.

## Paso 4 · `res/drawable/fondo_boton_redondeado.xml`

Consigna 6: drawable XML de tipo `shape` con esquinas redondeadas.

```
<?xml version="1.0" encoding="utf-8"?>
<shape xmlns:android="http://schemas.android.com/apk/res/android"
  android:shape="rectangle">

  <solid android:color="@color/primario" />

  <corners android:radius="@dimen/radio_esquina" />

  <stroke
    android:width="@dimen/borde_boton"
    android:color="@color/primario_oscuro" />

  <padding
    android:left="@dimen/espaciado_medio"
    android:right="@dimen/espaciado_medio"
    android:top="@dimen/espaciado_pequeno"
    android:bottom="@dimen/espaciado_pequeno" />
</shape>
```

| Etiqueta                      | Función                                                                                        |
|-------------------------------|------------------------------------------------------------------------------------------------|
| <code>android:shape</code>    | <code>rectangle</code> , <code>oval</code> , <code>line</code> o <code>ring</code>             |
| <code>&lt;solid&gt;</code>    | Color de relleno                                                                               |
| <code>&lt;corners&gt;</code>  | Radio de esquinas; admite <code>topLeftRadius</code> etc. para radios distintos por esquina    |
| <code>&lt;stroke&gt;</code>   | Grosor y color del borde; con <code>dashWidth</code> / <code>dashGap</code> se vuelve punteado |
| <code>&lt;gradient&gt;</code> | Degradado lineal, radial o de barrido                                                          |
| <code>&lt;padding&gt;</code>  | Relleno interno que el drawable impone a la vista que lo usa                                   |

## POR QUÉ UN SHAPE XML Y NO UN PNG

Un `shape` es una instrucción de dibujo, no una imagen. Pesa unos 400 bytes, se redibuja nítido en cualquier densidad, no necesita variantes `-hdpi` / `-xxhdpi`, y sus colores son referencias a `colors.xml`, de modo que cambian solos en modo oscuro. Un PNG equivalente pesaría decenas de kilobytes, exigiría seis variantes de densidad y quedaría congelado en su color.

## Variante opcional con estados ( `selector` )

Para que el botón reaccione visualmente a la pulsación, sin una línea de Java:

```
<?xml version="1.0" encoding="utf-8"?>
<!-- res/drawable/fondo_boton_estados.xml -->
<selector xmlns:android="http://schemas.android.com/apk/res/android">
  <item android:state_pressed="true">
    <shape android:shape="rectangle">
      <solid android:color="@color/primario_oscuro" />
      <corners android:radius="@dimen/radio_esquina" />
    </shape>
  </item>
  <item>
    <shape android:shape="rectangle">
      <solid android:color="@color/primario" />
      <corners android:radius="@dimen/radio_esquina" />
    </shape>
  </item>
</selector>
```

El orden importa: el sistema recorre los `<item>` de arriba abajo y aplica **el primero que coincida**. El `<item>` sin condiciones debe ir siempre al final, porque coincide con todo.

## Paso 5 • `res/drawable/ic_agenda.xml` (vector)

```
<?xml version="1.0" encoding="utf-8"?>
<vector xmlns:android="http://schemas.android.com/apk/res/android"
  android:width="@dimen/tamano_logo"
  android:height="@dimen/tamano_logo"
  android:viewportWidth="24"
  android:viewportHeight="24">

  <!-- Cuerpo de la agenda -->
  <path
    android:pathData="M4,6 H20 V20 H4 Z"
    android:fillColor="@android:color/transparent"
    android:strokeColor="@color/primario"
    android:strokeWidth="1.6" />

  <!-- Línea del encabezado -->
  <path
    android:pathData="M4,10 H20"
    android:strokeColor="@color/primario"
    android:strokeWidth="1.6" />

  <!-- Anillas superiores -->
  <path
    android:pathData="M8,3 V7 M16,3 V7"
    android:strokeColor="@color/primario"
    android:strokeWidth="1.6"
    android:strokeLineCap="round" />

  <!-- Día marcado -->
  <path
    android:pathData="M7,13 H11 V16 H7 Z"
    android:fillColor="@color/primario" />
</vector>
```

### Cómo leer un `pathData`

El atributo `pathData` usa la misma sintaxis que SVG. Mayúscula = coordenada absoluta; minúscula = relativa al punto actual.

| Comando | Significado                         | En el ejemplo                  |
|---------|-------------------------------------|--------------------------------|
| M x,y   | Move to: mover el lápiz sin dibujar | M4,6 → ir al punto (4, 6)      |
| H x     | Horizontal line: línea horizontal   | H20 → línea hasta x = 20       |
| V y     | Vertical line: línea vertical       | V20 → línea hasta y = 20       |
| L x,y   | Line to: línea al punto             | —                              |
| C ...   | Curva Bézier cúbica                 | —                              |
| Z       | Close: cerrar la figura             | Z → vuelve al inicio del trazo |

`viewportWidth` y `viewportHeight` definen el **sistema de coordenadas interno** (aquí, una cuadrícula de 24 × 24), mientras que `android:width` y `android:height` definen el **tamaño de dibujo real**. Son independientes: el mismo vector de 24 × 24 unidades se dibuja perfectamente a 24 dp o a 120 dp.

## Paso 6 • `res/values/themes.xml`

Consigna 5: un estilo propio llamado `EstiloBotonPrincipal`.

```
<?xml version="1.0" encoding="utf-8"?>
<resources xmlns:tools="http://schemas.android.com/tools">

    <!-- Tema base de la aplicación -->
    <style name="Theme.MiAgendaINTJEM" parent="Theme.AppCompat.DayNight.NoActionBar">
        <item name="colorPrimary">@color/primario</item>
        <item name="colorPrimaryDark">@color/primario_oscuro</item>
        <item name="colorAccent">@color/acento</item>
        <item name="android:colorBackground">@color/superficie</item>
        <item name="android:textColorPrimary">@color/texto_principal</item>
        <item name="android:statusBarColor" tools:targetApi="21">@color/primario_oscuro</item>
    </style>

    <!-- Estilo reutilizable de botón principal -->
    <style name="EstiloBotonPrincipal" parent="Widget.AppCompat.Button">
        <item name="android:background">@drawable/fondo_boton_redondeado</item>
        <item name="android:textColor">@color/texto_sobre_primario</item>
        <item name="android:textSize">@dimen/texto_boton</item>
        <item name="android:minHeight">@dimen/altura_boton</item>
        <item name="android:paddingStart">@dimen/espaciado_grande</item>
        <item name="android:paddingEnd">@dimen/espaciado_grande</item>
        <item name="android:textAllCaps">false</item>
        <item name="android:elevation">@dimen/elevacion_boton</item>
    </style>
</resources>
```

## ESTILO FRENTE A TEMA: LA DIFERENCIA EXACTA

Un **estilo** es un conjunto de atributos que se aplica a **una vista**, mediante `style="@style/..."` en el XML.

Un **tema** es un estilo aplicado a **toda una Activity o a toda la aplicación**, declarado en el `AndroidManifest.xml`. Sus atributos quedan disponibles para cualquier vista que los consulte con `attr/`, y se heredan hacia abajo en la jerarquía.

Técnicamente ambos se declaran con la misma etiqueta `<style>`. La diferencia es **dónde se aplican**, no cómo se escriben. Esta pregunta aparece con mucha frecuencia en sustentación.

## Convención de herencia por punto

Existen dos formas de heredar un estilo:

```
<!-- 1. Por atributo parent: obligatorio si el padre es de una biblioteca externa -->
<style name="EstiloBotonPrincipal" parent="Widget.AppCompat.Button">

<!-- 2. Por notación de punto: solo válido para estilos PROPIOS -->
<style name="EstiloBotonPrincipal.Secundario">
    <item name="android:background">@drawable/fondo_boton_secundario</item>
</style>
```

`EstiloBotonPrincipal.Secundario` hereda automáticamente todo lo de `EstiloBotonPrincipal` y sobrescribe solo el fondo. Es la forma idiomática de declarar variantes.

## Registro del tema en el manifiesto

```
<application
    android:name=".MiAgendaApp"
    android:icon="@mipmap/ic_launcher"
    android:label="@string/app_name"
    android:roundIcon="@mipmap/ic_launcher_round"
    android:supportRtl="true"
    android:theme="@style/Theme.MiAgendaINTJEM">

    <activity
        android:name=".MainActivity"
        android:exported="true">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
    </activity>
</application>
```

Obsérvese que incluso el manifiesto referencia recursos: `@string/app_name`, `@mipmap/ic_launcher`, `@style/Theme.MiAgendaINTJEM`. La externalización llega hasta ahí.

## Paso 7 • `res/layout/activity_main.xml`

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/raizPrincipal"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/superficie"
    android:padding="@dimen/espaciado_grande"
    tools:context=".MainActivity">

    <!-- 1. ENCABEZADO -->
    <TextView
        android:id="@+id/tvEncabezado"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:gravity="center"
        android:text="@string/titulo_encabezado"
        android:textColor="@color/primario"
        android:textSize="@dimen/texto_titulo"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <!-- 2. IMAGEN -->
    <ImageView
        android:id="@+id/ivLogo"
        android:layout_width="@dimen/tamano_logo"
        android:layout_height="@dimen/tamano_logo"
        android:layout_marginTop="@dimen/espaciado_grande"
        android:contentDescription="@string/desc_logo"
        android:src="@drawable/ic_agenda"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/tvEncabezado" />

    <!-- 3. CAMPO DE BIENVENIDA -->
    <TextView
        android:id="@+id/tvBienvenida"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="@dimen/espaciado_grande"
        android:gravity="center"
        android:padding="@dimen/espaciado_medio"
        android:text="@string/mensaje_bienvenida"
        android:textColor="@color/texto_principal"
        android:textSize="@dimen/texto_cuerpo"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/ivLogo" />

    <!-- 4. BOTONES EN CADENA HORIZONTAL -->
    <Button
        android:id="@+id/btnIngresar"
        style="@style/EstiloBotonPrincipal"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="@dimen/espaciado_grande"
        android:layout_marginEnd="@dimen/espaciado_pequeno"
        android:text="@string/boton_ingresar"
        app:layout_constraintEnd_toStartOf="@id/btnAcercaDe"
        app:layout_constraintHorizontal_chainStyle="spread"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toBottomOf="@id/tvBienvenida" />

    <Button
        android:id="@+id/btnAcercaDe"
        style="@style/EstiloBotonPrincipal"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="@dimen/espaciado_pequeno"
        android:text="@string/boton_acerca_de"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/btnIngresar"
        app:layout_constraintTop_toTopOf="@id/btnIngresar" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

## Qué hay que saber explicar de este layout

1. **Cero literales.** Todo texto, color, dimensión y gráfico es una referencia. Es el criterio de aprobación de la actividad.
2. **0dp en los anchos** significa `MATCH_CONSTRAINT`, no cero. El ancho lo determinan las restricciones.
3. **La cadena horizontal** entre `btnIngresar` y `btnAcercaDe`: cada uno apunta al otro (`End_toStartOf` / `Start_toEndOf`) y los extremos apuntan al contenedor. `chainStyle="spread"` reparte el espacio en partes iguales. Los botones se reparten el ancho automáticamente en cualquier pantalla.
4. **contentDescription en la imagen** es obligatorio para lectores de pantalla (TalkBack). Sin él, Lint emite la advertencia `ContentDescription`. Si una imagen fuese puramente decorativa, lo correcto sería `android:importantForAccessibility="none"`, no omitir el atributo.
5. **El espacio de nombres tools:** no se compila en el APK: solo alimenta la vista previa del editor. `tools:context` le dice al editor qué tema aplicar; `tools:text="Ejemplo"` permite ver texto de muestra sin que aparezca en ejecución.

## Paso 8 • MainActivity.java

```

package com.example.miagendaintjem;

import android.os.Bundle;
import android.widget.Button;
import android.widget.TextView;
import android.widget.Toast;

import androidx.appcompat.app.AppCompatActivity;
import androidx.core.content.ContextCompat;

public class MainActivity extends AppCompatActivity {

    private TextView tvBienvenida;
    private Button btnIngresar;
    private Button btnAcercaDe;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);    // recurso de layout

        enlazarVistas();
        aplicarRecursosDesdeJava();
        configurarEventos();
    }

    private void enlazarVistas() {
        tvBienvenida = findViewById(R.id.tvBienvenida);
        btnIngresar = findViewById(R.id.btnIngresar);
        btnAcercaDe = findViewById(R.id.btnAcercaDe);
    }

    /**
     * Demuestra las tres formas de leer recursos desde Java.
     * Ninguna línea contiene un literal de interfaz.
     */
    private void aplicarRecursosDesdeJava() {
        String saludo = getString(R.string.mensaje_bienvenida);
        int colorTexto = ContextCompat.getColor(this, R.color.texto_principal);
        int relleno = getResources().getDimensionPixelSize(R.dimen.espaciado_medio);

        tvBienvenida.setText(saludo);
        tvBienvenida.setTextColor(colorTexto);
        tvBienvenida.setPadding(relleno, relleno, relleno, relleno);
    }

    private void configurarEventos() {
        btnIngresar.setOnClickListener(v ->
            Toast.makeText(this, R.string.msg_ingresar, Toast.LENGTH_SHORT).show());

        btnAcercaDe.setOnClickListener(v -> {
            // Cadena con parámetro posicional resuelto en tiempo de ejecución
            String mensaje = getString(R.string.msg_acerca_de, getString(R.string.app_name));
            Toast.makeText(this, mensaje, Toast.LENGTH_LONG).show();
        });
    }
}

```

#### MÉTODO OBSOLETO QUE TODAVÍA CIRCULA EN TUTORIALES

`getResources().getColor(R.color.primario)` quedó **obsoleto en API 23**, porque no recibe el tema y por tanto no puede resolver colores dependientes del tema (modo oscuro incluido).

La forma correcta es `ContextCompat.getColor(context, R.color.primario)`, que internamente elige la implementación adecuada según la versión del sistema. Usar el método obsoleto en la entrega es un descuido fácil de detectar y fácil de evitar.

### Entregables de AP-3.7.1

- ☐ Proyecto comprimido o enlace al repositorio **con commits de ambos integrantes**.
- ☐ Captura de la pantalla ejecutada en el emulador.
- ☐ `colors.xml`, `dimens.xml` y `strings.xml` impresos en el anexo del informe.
- ☐ Indicación explícita de quién ejerció el rol de Conductor y quién el de Navegante.

## AP-3.7.2 · Adaptabilidad y calificadores

**Modalidad:** práctica · 90 min · en pareja

**Restricción de la consigna:** todo debe lograrse **sin modificar una sola línea de Java**. Ése es el punto pedagógico: demostrar que la adaptación es responsabilidad del sistema de recursos, no del código.

### Requerimiento 1 · `res/layout-land/activity_main.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:id="@+id/raizPrincipal"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:background="@color/superficie"
    android:padding="@dimen/espaciado_grande"
    tools:context=".MainActivity">

    <!-- Línea guía invisible que parte la pantalla al 40 % -->
    <androidx.constraintlayout.widget.Guideline
        android:id="@+id/guiaVertical"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        app:layout_constraintGuide_percent="0.4" />

    <!-- IZQUIERDA: imagen centrada verticalmente -->
    <ImageView
        android:id="@+id/ivLogo"
        android:layout_width="@dimen/tamano_logo"
        android:layout_height="@dimen/tamano_logo"
        android:contentDescription="@string/desc_logo"
        android:src="@drawable/ic_agenda"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toStartOf="@id/guiaVertical"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

    <!-- DERECHA: encabezado -->
    <TextView
        android:id="@+id/tvEncabezado"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:gravity="start"
        android:text="@string/titulo_encabezado"
        android:textColor="@color/primario"
        android:textSize="@dimen/texto_titulo"
        android:textStyle="bold"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/guiaVertical"
        app:layout_constraintTop_toTopOf="parent" />

    <TextView
        android:id="@+id/tvBienvenida"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="@dimen/espaciado_medio"
        android:gravity="start"
        android:text="@string/mensaje_bienvenida"
        android:textColor="@color/texto_principal"
        android:textSize="@dimen/texto_cuerpo"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/guiaVertical"
        app:layout_constraintTop_toBottomOf="@id/tvEncabezado" />

    <Button
        android:id="@+id/btnIngresar"
        style="@style/EstiloBotonPrincipal"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginTop="@dimen/espaciado_grande"
        android:layout_marginEnd="@dimen/espaciado_pequeno"
        android:text="@string/boton_ingresar"
        app:layout_constraintEnd_toStartOf="@id/btnAcercaDe"
        app:layout_constraintStart_toEndOf="@id/guiaVertical"
        app:layout_constraintTop_toBottomOf="@id/tvBienvenida" />

    <Button
        android:id="@+id/btnAcercaDe"
        style="@style/EstiloBotonPrincipal"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_marginStart="@dimen/espaciado_pequeno"
        android:text="@string/boton_acerca_de"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toEndOf="@id/btnIngresar"
        app:layout_constraintTop_toTopOf="@id/btnIngresar" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

#### EL ERROR QUE LA PROPIA GUÍA ANTICIPA, EXPLICADO

Los cinco identificadores — `ivLogo`, `tvEncabezado`, `tvBienvenida`, `btnIngresar`, `btnAcercaDe` — son **exactamente los mismos** que en `layout/activity_main.xml`. Solo cambian las restricciones.

**Por qué cerrarse la app si no coinciden:** al rotar, el sistema **destruye y recrea** la Activity. El nuevo `onCreate()` ejecuta el mismo `findViewById(R.id.btnIngresar)`. Si el layout horizontal llamó a ese botón `btnEntrar`, `findViewById` devuelve `null`, y la siguiente línea — `btnIngresar.setOnClickListener(...)` — lanza `NullPointerException` y la aplicación se cierra.

**Formulación para la defensa:** “Un calificador produce una *variante* de un mismo recurso, no un recurso distinto. El identificador en la clase `R` es único: `R.layout.activity_main` existe una sola vez. Por lo tanto el contrato con el código Java —el conjunto de `id` disponibles— debe ser idéntico en todas las variantes.”

## Requerimiento 2 • `res/values-en/strings.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <string name="titulo_encabezado">Academic Planner</string>
  <string name="mensaje_bienvenida">Welcome to the system. Keep your tasks, subjects and due dates in one place.</string>
  <string name="boton_ingresar">Sign in</string>
  <string name="boton_acerca_de">About</string>
  <string name="desc_logo">Academic planner icon</string>
  <string name="msg_ingresar">Opening your planner.</string>
  <string name="msg_acerca_de">%1$s · version 1.0 · Section 3.7</string>
</resources>
```

### TRES DETALLES QUE HAY QUE SABER JUSTIFICAR

1. **app\_name no se traduce.** El nombre propio de la aplicación se mantiene. Al no estar en `values-en/`, el sistema recurre automáticamente a `values/`. Esto demuestra que se entendió el mecanismo de respaldo.
2. **El parámetro `%1$s` se conserva.** Si el traductor lo borra, la aplicación lanza `MissingFormatArgumentException` en ejecución. Para protegerse, se marca `translatable="false"` las cadenas que no deben traducirse.
3. **El respaldo es por recurso, no por archivo.** Si `values-en/strings.xml` define seis cadenas y `values/strings.xml` define ocho, en inglés se usan las seis traducidas y las dos restantes en español. No se cae la aplicación.

**Verificación en el emulador:** Ajustes → Sistema → Idiomas → añadir *English (United States)* y moverlo al primer lugar. La aplicación se reinicia con la configuración nueva y muestra los textos en inglés, sin recompilar.

## Requerimiento 3 • `res/values-night/colors.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <!-- Mismos NOMBRES, distintos VALORES -->
  <color name="primario">#A3C9FF</color>
  <color name="primario_oscuro">#004884</color>
  <color name="acento">#4FD8C7</color>

  <color name="superficie">#121416</color>
  <color name="texto_principal">#E2E2E6</color>
  <color name="texto_sobre_primario">#00315D</color>

  <color name="error">#FFB4AB</color>
</resources>
```

### LA INVERSIÓN QUE CASI NADIE HACE BIEN

En modo oscuro el color primario **se aclara** (de `#0B5FA5` a `#A3C9FF`) y el texto que va sobre él **se oscurece** (de `#FFFFFF` a `#00315D`).

La razón es física: un color saturado y oscuro sobre fondo negro tiene muy poco contraste y además produce halos por aberración cromática. Material 3 formaliza esta inversión asignando a cada rol un *tono* distinto según el esquema (ver capítulo 7).

El error típico es copiar la paleta clara a `values-night` cambiando solo el fondo. El resultado compila, se ve “oscuro” y es ilegible.

**Condición necesaria:** el tema debe heredar de una variante **DayNight** (`Theme.AppCompat.DayNight.*` o `Theme.Material3.DayNight.*`). Es esa herencia la que hace que la biblioteca de compatibilidad propague al `Configuration` la bandera de modo noche del sistema. Con un tema que no sea `DayNight`, la carpeta `values-night` nunca se selecciona, aunque exista.

**Verificación:** Ajustes → Pantalla → Tema oscuro. La aplicación cambia sin reiniciarse y sin Java.

## Requerimiento 4 • Drawable vectorial y nitidez en dos densidades

El vector `ic_agenda.xml` del capítulo anterior ya cumple el requisito. Lo que la actividad pide **demostrar** es la diferencia frente a un mapa de bits.

### Experimento propuesto (queda muy bien en el informe)

1. Colocar un PNG de 48 × 48 px **únicamente** en `res/drawable-mdpi/logo_bitmap.png`.
2. Crear un layout de prueba que muestre, lado a lado, `@drawable/ic_agenda` y `@drawable/logo_bitmap`, ambos a 120 dp.
3. Ejecutar en un emulador **mdpi** (por ejemplo Nexus S) → ambos se ven aceptables.
4. Ejecutar en un emulador **xxhdpi** (por ejemplo Pixel 6) → el PNG aparece visiblemente borroso; el vector sigue nítido.
5. Capturar ambas pantallas y colocarlas en el informe una junto a otra.

**Explicación del resultado:** la densidad es el único calificador que el sistema **nunca descarta**. Al no encontrar `drawable-xxhdpi`, toma el `mdpi` y lo **escala por interpolación** ×3, inventando píxeles que no existían. El vector, en cambio, no se escala: se **rasteriza de nuevo** a partir de sus instrucciones geométricas, a la resolución exacta que hace falta.

| Criterio                   | Vector XML                                     | Mapa de bits PNG        |
|----------------------------|------------------------------------------------|-------------------------|
| Archivos necesarios        | 1                                              | Hasta 6 (mdpi...xxhdpi) |
| Peso típico                | 0,5-2 KB                                       | 20-200 KB por variante  |
| Nitidez al ampliar         | Perfecta siempre                               | Se degrada              |
| Color modificable por tema | Sí ( <code>@color</code> , <code>tint</code> ) | No sin filtros          |
| Apto para fotografías      | <b>No</b>                                      | Sí                      |
| Costo en CPU               | Rasteriza al inflar                            | Solo decodifica         |

La última fila es la contrapartida honesta: un vector con cientos de trazos es más caro de dibujar que un PNG. Para íconos, el vector gana; para fotografías, es inviable.

## Requerimiento 5 • `res/values-sw600dp/dimens.xml`

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <!-- Márgenes ampliados para pantallas de 600dp o más de ancho mínimo -->
  <dimen name="espaciado_pequeno">12dp</dimen>
  <dimen name="espaciado_medio">24dp</dimen>
  <dimen name="espaciado_grande">48dp</dimen>

  <dimen name="tamano_logo">200dp</dimen>

  <dimen name="texto_titulo">32sp</dimen>
  <dimen name="texto_cuerpo">18sp</dimen>
</resources>
```

#### QUÉ SIGNIFICA EXACTAMENTE “SMALLESTWIDTH”

sw600dp = “el **menor** de los dos lados de la pantalla mide al menos 600 dp”.

La clave está en *menor*: es el lado corto, y por eso **no cambia al rotar**. Un teléfono típico tiene un ancho mínimo de 360 dp tanto en vertical como en horizontal; una tablet de 7” ronda los 600 dp y una de 10” los 720 dp.

Por eso sw600dp significa de hecho “esto es una tablet”, y por eso tiene precedencia superior a land: describe el **hardware**, que es estable, no la **postura**, que es volátil.

sw600dp sustituye a los antiguos calificadores small/normal/large/xlarge, obsoletos desde API 13 por ser imprecisos.

**No hace falta declarar las once dimensiones** en la variante: solo las que cambian. Las demás se resuelven por respaldo contra values/dimens.xml.

## Verificación en clase: la demostración

La consigna exige demostrar las tres cosas **en un mismo APK en ejecución**. Guion sugerido, de 90 segundos:

1. Abrir la aplicación en vertical. Señalar el layout.
2. Rotar el dispositivo → aparece la variante layout-land, con los mismos id. “No hemos escrito código para esto; el sistema eligió la carpeta.”
3. Ajustes → Idioma → English → volver a la app. Los textos cambian. “La app no se recompiló; se reinició con otro Locale y resources.arsc devolvió otras filas.”
4. Ajustes → Tema oscuro. La paleta cambia. “Los nombres de color son los mismos; los valores vienen de values-night.”
5. Cambiar al emulador de tablet → márgenes ampliados por sw600dp.

#### FRASE QUE CIERRA LA DEMOSTRACIÓN

“Cuatro adaptaciones distintas, cero líneas de Java. La lógica de adaptación no la escribimos nosotros: la escribió el equipo de Android, y nosotros solo declaramos en qué condiciones aplicarla.”



```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <color name="md_primary">#A3C9FF</color>
  <color name="md_on_primary">#00315D</color>
  <color name="md_primary_container">#004884</color>
  <color name="md_on_primary_container">#D3E4FF</color>

  <color name="md_secondary">#B7C3D6</color>
  <color name="md_on_secondary">#21303C</color>
  <color name="md_secondary_container">#374653</color>
  <color name="md_on_secondary_container">#D3E4F5</color>

  <color name="md_surface">#121416</color>
  <color name="md_on_surface">#E2E2E6</color>
  <color name="md_surface_variant">#43474E</color>
  <color name="md_on_surface_variant">#C3C6CF</color>
  <color name="md_outline">#8D9199</color>

  <color name="md_error">#FFB4AB</color>
  <color name="md_on_error">#690005</color>
  <color name="md_error_container">#93000A</color>
  <color name="md_on_error_container">#FFDAD6</color>
</resources>
```

La regla “on” que estructura todo el sistema

LA IDEA CENTRAL DE MATERIAL 3, EN UNA LÍNEA

Los roles vienen **en pares**. Todo color de fondo tiene su color de contenido: `colorPrimary` ↔ `colorOnPrimary`, `colorSurface` ↔ `colorOnSurface`, `colorError` ↔ `colorOnError`.

El prefijo “on” significa literalmente “lo que va encima de”. Si un elemento tiene fondo `colorPrimary`, su texto e íconos **deben** usar `colorOnPrimary`. Ese par está calculado de fábrica para cumplir el contraste mínimo.

**Consecuencia práctica:** respetando los pares, la accesibilidad cromática viene garantizada. Si en algún punto hace falta “buscar un color que contraste”, es señal de que se rompió un par.

Requerimiento 3 · Derivación de tonos con Material Theme Builder

Ésta es la parte que la consigna pide **explicar en el informe**, y es donde se marca la diferencia.

El espacio de color HCT

Material 3 no razona en RGB ni en HSL, sino en **HCT** (*Hue, Chroma, Tone*), un espacio creado por Google que combina el tono y el croma de CAM16 con la claridad **L\*** de CIELAB.

- Hue (matiz):** 0–360°, el “color” propiamente dicho.
- Chroma (croma):** cuán saturado es.
- Tone (tono):** 0 = negro absoluto, 100 = blanco absoluto.

La propiedad decisiva es que **Tone está calibrado contra la percepción humana de luminancia**, de modo que la diferencia de tono entre dos colores predice su contraste. Material 3 lo aprovecha: separaciones de tono suficientemente grandes garantizan contrastes accesibles por construcción.

De un color de marca a cinco paletas

A partir de un único color semilla (el azul institucional `#0B5FAS`), el Material Theme Builder genera **cinco paletas tonales**:

| Paleta          | Cómo se deriva de la semilla           | Para qué sirve                            |
|-----------------|----------------------------------------|-------------------------------------------|
| Primary         | Mismo matiz, croma elevado             | Acciones principales                      |
| Secondary       | Mismo matiz, croma reducido            | Acciones de apoyo, menos llamativas       |
| Tertiary        | Matiz desplazado (≈ +60°), croma medio | Acentos y contraste expresivo             |
| Neutral         | Mismo matiz, croma casi nulo           | Fondos y superficies                      |
| Neutral Variant | Mismo matiz, croma bajo                | Contornos y superficies diferenciadas     |
| Error           | Matiz rojo fijo, croma alto            | Estados de error (no depende de la marca) |

Cada paleta se despliega en **trece tonos fijos**: 0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 95, 99, 100.

Asignación de tonos a roles

Ésta es la tabla que responde la consigna “explicar cómo se derivan los tonos restantes”:

| Rol                                  | Tema claro                 | Tema oscuro                |
|--------------------------------------|----------------------------|----------------------------|
| <code>colorPrimary</code>            | Primary <b>T40</b>         | Primary <b>T80</b>         |
| <code>colorOnPrimary</code>          | Primary <b>T100</b>        | Primary <b>T20</b>         |
| <code>colorPrimaryContainer</code>   | Primary <b>T90</b>         | Primary <b>T30</b>         |
| <code>colorOnPrimaryContainer</code> | Primary <b>T10</b>         | Primary <b>T90</b>         |
| <code>colorSecondary</code>          | Secondary <b>T40</b>       | Secondary <b>T80</b>       |
| <code>colorSurface</code>            | Neutral <b>T99</b>         | Neutral <b>T10</b>         |
| <code>colorOnSurface</code>          | Neutral <b>T10</b>         | Neutral <b>T90</b>         |
| <code>colorOutline</code>            | Neutral Variant <b>T50</b> | Neutral Variant <b>T60</b> |
| <code>colorError</code>              | Error <b>T40</b>           | Error <b>T80</b>           |

## TEXTO LISTO PARA EL INFORME

“El esquema no se elige color por color: se deriva. A partir de un color semilla se calcula su matiz y croma en el espacio HCT y se generan cinco paletas tonales de trece tonos cada una. Los roles se asignan por posición fija dentro de esas paletas: en tema claro, `colorPrimary` es el tono 40 y `colorOnPrimary` el tono 100; en tema oscuro esa relación se invierte a 80 y 20. Como el tono de HCT está calibrado contra la luminancia percibida, la distancia tonal fija entre los pares garantiza el contraste sin necesidad de verificarlo caso por caso. Nuestra verificación WCAG confirma empíricamente esa garantía teórica.”

**Procedimiento operativo:** entrar en el Material Theme Builder, introducir `#0B5FA5` como *source color*, exportar en formato *Android Views (XML)* y pegar los archivos generados. Los nombres exportados (`md_theme_light_primary`, etc.) pueden simplificarse, pero los **roles** deben conservarse.

## Requerimiento 4 · Tres `ShapeAppearance` propios

```
<!-- res/values/themes.xml -->
<style name="ShapeAppearance.MiAgenda.Small"
    parent="ShapeAppearance.Material3.SmallComponent">
    <item name="cornerFamily">rounded</item>
    <item name="cornerSize">@dimen/radio_pequeno</item>
</style>

<style name="ShapeAppearance.MiAgenda.Medium"
    parent="ShapeAppearance.Material3.MediumComponent">
    <item name="cornerFamily">rounded</item>
    <item name="cornerSize">@dimen/radio_medio</item>
</style>

<style name="ShapeAppearance.MiAgenda.Large"
    parent="ShapeAppearance.Material3.LargeComponent">
    <item name="cornerFamily">rounded</item>
    <item name="cornerSize">@dimen/radio_grande</item>
</style>
```

```
<!-- res/values/dimens.xml -->
<dimen name="radio_pequeno">8dp</dimen>
<dimen name="radio_medio">16dp</dimen>
<dimen name="radio_grande">28dp</dimen>
```

`cornerFamily` admite `rounded` (esquina circular) o `cut` (esquina achaflanada). Aplicar `cut` a la forma grande es una manera barata de dar personalidad visual a la marca.

**Qué componente usa cada forma:** los componentes Material declaran de fábrica a qué categoría pertenecen. Los botones y campos de texto usan la forma *small*; las tarjetas (`MaterialCardView`, que se usará en la guía 3.8) usan la *medium*; los diálogos y hojas inferiores usan la *large*. Al redefinir las tres en el tema, **todos** los componentes futuros cambian de forma a la vez.

## Requerimiento 5 · Escala tipográfica con `TextAppearance`

### Los quince roles de Material 3

| Categoría | Roles                  | Tamaño base (sp) | Uso                                   |
|-----------|------------------------|------------------|---------------------------------------|
| Display   | Large / Medium / Small | 57 / 45 / 36     | Números y títulos enormes, uso escaso |
| Headline  | Large / Medium / Small | 32 / 28 / 24     | Títulos de pantalla                   |
| Title     | Large / Medium / Small | 22 / 16 / 14     | Títulos de sección y de tarjeta       |
| Body      | Large / Medium / Small | 16 / 14 / 12     | Texto corrido                         |
| Label     | Large / Medium / Small | 14 / 12 / 11     | Botones, pestañas, etiquetas          |

### Definición

```
<style name="TextAppearance.MiAgenda.HeadlineSmall"
    parent="TextAppearance.Material3.HeadlineSmall">
    <item name="android:textSize">@dimen/texto_headline_small</item>
    <item name="android:fontFamily">sans-serif-medium</item>
    <item name="android:letterSpacing">0</item>
</style>

<style name="TextAppearance.MiAgenda.BodyLarge"
    parent="TextAppearance.Material3.BodyLarge">
    <item name="android:textSize">@dimen/texto_body_large</item>
</style>

<style name="TextAppearance.MiAgenda.LabelLarge"
    parent="TextAppearance.Material3.LabelLarge">
    <item name="android:textSize">@dimen/texto_label_large</item>
    <item name="android:textAllCaps">>false</item>
</style>
```

### Registro en el tema y uso en el layout

```
<style name="Theme.MiAgendaINTJEM" parent="Theme.Material3.DayNight.NoActionBar">
    <item name="textAppearanceHeadlineSmall">@style/TextAppearance.MiAgenda.HeadlineSmall</item>
    <item name="textAppearanceBodyLarge">@style/TextAppearance.MiAgenda.BodyLarge</item>
    <item name="textAppearanceLabelLarge">@style/TextAppearance.MiAgenda.LabelLarge</item>
</style>
```

```
<!-- En el layout: se declara el ROL, no el tamaño -->
<TextView
    android:id="@+id/tvEncabezado"
    android:textAppearance="?attr/textAppearanceHeadlineSmall"
    android:textColor="?attr/colorPrimary"
    ... />
```

LOS TRES NIVELES DE EXTERNALIZACIÓN, EN ORDEN DE MADUREZ

**Nivel 0 (prohibido):** `android:textSize="24sp"` — literal.

**Nivel 1 (guía 3.7 básica):** `android:textSize="@dimen/texto_titulo"` — el valor se externaliza, pero cada vista sigue decidiendo su tamaño.

**Nivel 2 (Material 3):** `android:textAppearance="?attr/textAppearanceHeadlineSmall"` — la vista declara *qué papel* cumple su texto y el tema decide tamaño, familia, peso e interletraje a la vez.

El nivel 2 es cualitativamente distinto: cambiar la identidad tipográfica de toda la aplicación pasa a ser una edición en un archivo, no una búsqueda de `textSize` por todo el proyecto.

Requerimiento 6 · Verificación del tema oscuro automático

No hay nada que programar. La combinación de dos hechos lo resuelve:

- 1. El tema hereda de `Theme.Material3.**DayNight*.NoActionBar`.
- 2. Existe `res/values-night/colors.xml` con los mismos nombres.

Cuando el usuario activa el tema oscuro, el sistema modifica la bandera `uiMode` del `Configuration`, lo cual hace que el calificador `night` empiece a coincidir. La misma referencia `?attr/colorSurface` devuelve otro valor. Cero líneas de Java.

Si se quisiera forzar un modo (por ejemplo desde un ajuste dentro de la app), la forma correcta sería:

```
AppCompatActivity.setDefaultNightMode(AppCompatActivity.MODE_NIGHT_YES);
```

Conviene mencionarlo en la sustentación **aclarando que no se usó**, porque la consigna pide explícitamente que el cambio ocurra sin lógica en Java.

Requerimiento 7 · Verificación de contraste WCAG

La fórmula

Contraste = 
$$\frac{L_{\text{claro}} + 0.05}{L_{\text{oscuro}} + 0.05}$$

donde **L** es la luminancia relativa de cada color:

$$L = 0.2126 \cdot R + 0.7152 \cdot G + 0.0722 \cdot B$$

siendo, para cada canal  $c \in \{R, G, B\}$  normalizado a  $[0,1]$ :

$$c \leq 0.03928 \rightarrow c / 12.92$$

$$c > 0.03928 \rightarrow ((c + 0.055) / 1.055) ^ {2.4}$$

Los coeficientes 0.2126 / 0.7152 / 0.0722 reflejan que el ojo humano es mucho más sensible al verde que al azul.

Umbrales de WCAG 2.1 nivel AA

| Elemento                                     | Mínimo exigido |
|----------------------------------------------|----------------|
| Texto normal                                 | 4.5 : 1        |
| Texto grande (≥ 18 pt, o ≥ 14 pt en negrita) | 3.0 : 1        |
| Componentes de interfaz y gráficos           | 3.0 : 1        |

Tabla de verificación (valores calculados)

La consigna pide al menos tres pares. Se entregan nueve, más dos contraejemplos.

| #  | Color de texto                                     | Color de fondo                                  | Contraste | Resultado |
|----|----------------------------------------------------|-------------------------------------------------|-----------|-----------|
| 1  | <code>md_on_primary</code> #FFFFFF                 | <code>md_primary</code> #0B5FA5                 | 6.57 : 1  | Cumple    |
| 2  | <code>md_on_surface</code> #1A1C1E                 | <code>md_surface</code> #FDFCFE                 | 16.72 : 1 | Cumple    |
| 3  | <code>md_on_primary_container</code> #001C38       | <code>md_primary_container</code> #D3E4FF       | 13.33 : 1 | Cumple    |
| 4  | <code>md_on_error</code> #FFFFFF                   | <code>md_error</code> #BA1A1A                   | 6.46 : 1  | Cumple    |
| 5  | <code>md_on_secondary</code> #FFFFFF               | <code>md_secondary</code> #4F5B6B               | 6.91 : 1  | Cumple    |
| 6  | <code>night md_on_surface</code> #E2E2E6           | <code>night md_surface</code> #121416           | 14.29 : 1 | Cumple    |
| 7  | <code>night md_on_primary</code> #00315D           | <code>night md_primary</code> #A3C9FF           | 7.74 : 1  | Cumple    |
| 8  | <code>night md_on_primary_container</code> #D3E4FF | <code>night md_primary_container</code> #004884 | 7.22 : 1  | Cumple    |
| 9  | <code>night md_on_error</code> #690005             | <code>night md_error</code> #FFB4AB             | 7.72 : 1  | Cumple    |
| C1 | #FF5722 (color del fragmento auditado en AT-3.7.2) | #FFFFFF                                         | 3.16 : 1  | No cumple |
| C2 | #FFFFFF                                            | #4A90D9 (azul claro mal elegido)                | 3.34 : 1  | No cumple |

CÓMO PRESENTAR ESTA TABLA

Incluir los **dos contraejemplos** es lo que convierte una tabla de relleno en una verificación real. Demuestra que se midió y no se supuso, y permite cerrar con una observación fuerte:

“Los nueve pares generados por Material 3 cumplen con margen. Los dos que fallan son precisamente colores elegidos *a ojo*: el del fragmento auditado en la actividad teórica y un azul claro de aspecto agradable. Ésa es la razón práctica para derivar la paleta con el Theme Builder en lugar de escogerla manualmente.”

**Herramientas:** el propio Material Theme Builder muestra los contrastes; también sirven el *Contrast Checker* de WebAIM o el panel *Accessibility* del inspector

de Chrome.

Punto de verificación conceptual de la consigna

“La pareja debe explicar la diferencia entre nombrar un color por su apariencia ( azul\_claro ) y nombrarlo por su rol semántico ( colorPrimaryContainer ), y por qué la segunda forma es la que exige Material 3.”

Respuesta completa:

|                       | Nombre por apariencia                    | Nombre por rol                          |
|-----------------------|------------------------------------------|-----------------------------------------|
| Ejemplo               | azul_claro , naranja , blanco            | colorPrimaryContainer , colorOnSurface  |
| Qué describe          | El valor actual                          | La función dentro del sistema           |
| En modo oscuro        | El nombre miente ( blanco vale #121416 ) | El nombre sigue siendo cierto           |
| Al rediseñar la marca | Hay que renombrar en todo el proyecto    | No cambia nada                          |
| Quién decide el valor | El programador, en cada uso              | El tema, una sola vez                   |
| Verificable           | No                                       | Sí: los pares “on” garantizan contraste |

**Por qué Material 3 lo exige:** el sistema completo de Material 3 —temas dinámicos, modo oscuro, *Material You* tomando colores del fondo de pantalla del usuario— se apoya en que la aplicación **no sepa** qué color concreto se va a usar. Si la aplicación pide `?attr/colorPrimaryContainer`, el sistema puede sustituir ese valor por uno derivado del fondo de pantalla del usuario y la interfaz sigue siendo coherente y accesible. Si la aplicación pide `@color/azul_claro`, el sistema no tiene ningún punto donde intervenir. El nombre semántico **es la interfaz del contrato**; el nombre por apariencia lo rompe.

Entregables de AC-3.7.3

- ☐ **Lámina de sistema de diseño en una sola hoja**, con tres bloques: paleta (muestras de color con su nombre de rol y su hexadecimal, en claro y oscuro), escala tipográfica (los cinco roles usados, con su tamaño) y formas (tres rectángulos mostrando los radios small/medium/large).
- ☐ Capturas de la **misma** pantalla en modo claro y en modo oscuro, lado a lado.
- ☐ Tabla de verificación de contraste con los valores calculados y el resultado.

# Errores frecuentes y diagnóstico

Tabla de consulta rápida durante el laboratorio.

| Síntoma                                                                                        | Causa probable                                                                  | Solución                                                                 |
|------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| La app se cierra al rotar, con <code>NullPointerException</code>                               | Un <code>id</code> del layout horizontal no coincide con el del vertical        | Igualar los <code>android:id</code> en ambas variantes                   |
| Las vistas se apilan en la esquina superior izquierda al ejecutar, aunque el editor se ve bien | Falta alguna restricción horizontal o vertical                                  | Añadir restricciones en ambos ejes a cada vista                          |
| <code>values-night</code> no tiene ningún efecto                                               | El tema no hereda de una variante <code>DayNight</code>                         | Cambiar el <code>parent</code> a <code>Theme.Material3.DayNight.*</code> |
| <code>values-en</code> no tiene efecto                                                         | El idioma se cambió en el emulador pero no se movió al primer lugar de la lista | Reordenar el idioma en Ajustes                                           |
| Compila, pero los componentes Material se ven sin estilo o la app se cierra                    | Migración parcial: tema AppCompatActivity con componentes Material 3            | Revisar la cadena de herencia del tema                                   |
| <code>error: resource string/xxx not found</code>                                              | Nombre mal escrito, o recurso definido solo en una variante                     | Definir siempre el recurso en <code>values/</code> como respaldo         |
| <code>Error inflating class ...</code>                                                         | Dependencia faltante en <code>build.gradle</code> , o ruta de clase mal escrita | Verificar dependencias y el nombre completo de la clase en el XML        |
| El texto se solapa al subir la fuente del sistema                                              | Alturas fijas en dp junto a texto en sp                                         | Usar <code>wrap_content</code> en los contenedores de texto              |
| El PNG se ve borroso en el emulador nuevo                                                      | Solo existe la variante mdpi                                                    | Usar vector, o generar las variantes de densidad                         |
| La clase <code>R</code> aparece en rojo en todo el proyecto                                    | Un archivo XML de <code>res/</code> tiene un error de sintaxis                  | Revisar la pestaña <i>Build</i> : AAPT2 falló y no generó <code>R</code> |

### HERRAMIENTA QUE CONVIENE USAR ANTES DE ENTREGAR

**Analyze → Inspect Code** en Android Studio ejecuta Lint sobre el proyecto entero. Las reglas relevantes para este punto son:

- `HardcodedText` — texto literal en un layout.
- `SpUsage` — uso de `dp` donde correspondía `sp`.
- `PxUsage` — uso de `px`.
- `ContentDescription` — imagen sin descripción para accesibilidad.
- `MissingTranslation` — cadena sin traducir en alguna variante de idioma.
- `UnusedResources` — recursos declarados y nunca usados.

Adjuntar al informe una captura del reporte de Lint **sin advertencias** en estas categorías es una prueba objetiva de que la consigna “ningún valor literal” se cumplió, y vale más que cualquier afirmación.

# Guion de exposición

Duración objetivo: **8 minutos**. Los dos integrantes deben poder dar cualquiera de las dos partes, porque el docente pregunta a quien quiera.

## Minuto 0:00 – 1:00 · Apertura conceptual

“El punto 3.7 trata sobre recursos de interfaz. La idea central es una sola: una aplicación Android bien construida no escribe pantallas, declara recursos. El layout y el código Java nunca contienen el valor; contienen una referencia. Quien elige el valor concreto es el sistema operativo, en tiempo de ejecución, según el dispositivo, el idioma, la orientación y las preferencias de accesibilidad del usuario. Todo lo que mostraremos es consecuencia de esa decisión de diseño.”

## Minuto 1:00 – 2:30 · Cómo funciona el mecanismo

“AAPT2 compila el directorio `res/` y produce dos cosas: la clase `R`, que es un diccionario de identificadores enteros, y `resources.arsc`, una tabla binaria que asocia cada identificador con todas sus variantes. En ejecución, `getString(R.string.app_name)` no devuelve un texto guardado en el código: toma el entero, busca en la tabla y devuelve la fila que mejor coincide con la `Configuration` actual. Por eso `R.string.saludo` es un `int`, no un `String`.”

Mostrar en pantalla la clase `R` generada (Android Studio la abre con doble mayúscula → “`R.java`”).

## Minuto 2:30 – 4:00 · Demostración en vivo

Ejecutar la aplicación y realizar, sin tocar el código:

1. **Rotación** → aparece `layout-land`.
2. **Cambio de idioma** → aparece `values-en`.
3. **Modo oscuro** → aparece `values-night`.
4. **Emulador de tablet** → aparecen los márgenes de `values-sw600dp`.

“Cuatro adaptaciones, cero líneas de Java. Los cuatro calificadores son variantes del mismo recurso: el identificador en `R` sigue siendo único.”

## Minuto 4:00 – 5:30 · El sistema de diseño (si se hizo AC-3.7.3)

“Sobre esa base construimos el sistema de diseño con Material Design 3. Partimos de un único color semilla, el azul institucional. El Material Theme Builder lo convierte a HCT —matiz, croma y tono— y genera cinco paletas tonales de trece tonos. Los roles se asignan por posición fija: en tema claro `colorPrimary` es el tono 40 y `colorOnPrimary` el tono 100; en tema oscuro la relación se invierte a 80 y 20. Como el tono de HCT está calibrado contra la luminancia percibida, esa distancia fija garantiza el contraste.”

Mostrar la lámina del sistema de diseño.

## Minuto 5:30 – 6:30 · Verificación objetiva

“No lo dimos por supuesto: lo medimos. Verificamos nueve pares de color contra el mínimo de 4,5 a 1 que exige WCAG. Todos cumplen, el más ajustado con 6,46 a 1. Además incluimos dos contraejemplos elegidos a ojo: el naranja del fragmento que auditamos en la actividad teórica da 3,16 a 1 y no cumple. Ésa es la razón práctica para derivar la paleta en vez de escogerla.”

## Minuto 6:30 – 7:30 · La auditoría teórica

“En la actividad teórica auditamos un fragmento con siete fallas. La más grave no es el literal de texto sino `textSize` en `px`: el factor de escala de fuente del sistema solo interviene en la conversión de `sp` a píxeles. Al escribir `px`, el desarrollador fija el resultado final y elimina el punto donde el sistema podía intervenir. Un usuario con baja visión que configura la fuente al 200 % no percibe ningún cambio. No es un problema de estilo: es un incumplimiento del criterio WCAG 1.4.4.”

## Minuto 7:30 – 8:00 · Cierre

“En síntesis: externalizar convierte una decisión estática del programador en una decisión dinámica del sistema. El costo es declarar recursos con nombres semánticos; el beneficio es que adaptarse a un dispositivo, un idioma, un modo de color o una necesidad de accesibilidad nuevos deja de requerir código.”

### REGLAS DE SUSTENTACIÓN

1. **No leer**. Las diapositivas llevan palabras clave, no párrafos.
2. **Números concretos**: 4,5:1 · 160 dpi · 48 dp · 8 dp · API 24 · 3,16:1. Un número exacto vale más que tres adjetivos.
3. **Si no se sabe algo, decirlo**. “No lo verificamos” es infinitamente mejor que inventar: el docente pregunta en cadena y la segunda pregunta descubre la invención.
4. **Ambos deben saber todo**. La calificación se ajusta para la pareja completa si uno no puede explicar lo entregado.

# Banco de preguntas de defensa

Taparse la columna derecha y responder en voz alta. Si la respuesta no sale en diez segundos, no está aprendida.

## Sobre el sistema de recursos

| Pregunta                                                                 | Respuesta                                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ¿Qué es <code>R</code> ?                                                 | Una clase generada por AAPT2 con una constante <code>public static final int</code> por cada recurso. Es un diccionario de identificadores; no contiene valores.                                                                                       |
| ¿Dónde están los valores, entonces?                                      | En <code>resources.arsc</code> , una tabla binaria dentro del APK que asocia cada identificador con todas sus variantes de configuración.                                                                                                              |
| ¿Por qué <code>mipmap</code> está separado de <code>drawable</code> ?    | Porque al generar distribuciones por densidad se eliminan las carpetas <code>drawable-*dpi</code> ajenas al dispositivo, pero <code>mipmap-*</code> se conserva entera: el lanzador puede pedir el ícono en una densidad superior a la de la pantalla. |
| ¿Qué va en <code>mipmap</code> ?                                         | Solo <code>ic_launcher</code> . Cualquier otro gráfico va en <code>drawable</code> .                                                                                                                                                                   |
| Diferencia entre <code>@color/x</code> y <code>?attr/colorPrimary</code> | <code>@</code> referencia un recurso concreto; <code>?attr</code> referencia un atributo del <b>tema activo</b> , resuelto en ejecución. El segundo es el que hace posible el modo oscuro sin Java.                                                    |
| Diferencia entre <code>@+id/x</code> y <code>@id/x</code>                | <code>@+id</code> <b>crea</b> el identificador; <code>@id</code> <b>usa</b> uno ya existente.                                                                                                                                                          |
| ¿Qué hace exactamente un calificador?                                    | Produce una <b>variante</b> de un recurso ya existente. El identificador en <code>R</code> sigue siendo único.                                                                                                                                         |
| ¿Qué pasa si falta un recurso en <code>values-en</code> ?                | Se usa el de <code>values/</code> como respaldo. El respaldo es por recurso, no por archivo.                                                                                                                                                           |
| ¿Por qué <code>values/</code> debe estar siempre completo?               | Porque es el respaldo por omisión de todas las variantes.                                                                                                                                                                                              |

## Sobre unidades y accesibilidad

| Pregunta                                                                          | Respuesta                                                                                                           |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Fórmula de dp a px                                                                | <code>px = dp × (dpi / 160)</code>                                                                                  |
| ¿Qué es 160 dpi?                                                                  | La densidad de referencia (bucket mdpi), en la que 1 dp = 1 px.                                                     |
| Diferencia entre dp y sp                                                          | <code>sp</code> incluye además el factor <code>fontScale</code> configurado por el usuario. Se usa solo para texto. |
| ¿Qué pasa si uso <code>px</code> para texto y el usuario sube la fuente al 200 %? | Nada: <code>fontScale</code> no interviene en la conversión de <code>px</code> . Se incumple WCAG 1.4.4.            |
| ¿Cuánto mide <code>320px</code> en un xxhdp?                                      | <code>320 / 3 = 106,7 dp</code> . En mdpi serían 320 dp.                                                            |
| ¿Por qué 48 dp de altura mínima en un botón?                                      | Es el objetivo táctil mínimo recomendado, ≈ 9 mm, el área media de contacto de un dedo.                             |
| ¿Por qué múltiplos de 8 dp?                                                       | Es la retícula de espaciado de Material Design: una escala restringida produce ritmo visual coherente.              |
| ¿Cuándo es legítimo usar <code>px</code> ?                                        | Solo en manipulación de mapas de bits a nivel de píxel. Nunca en la interfaz.                                       |

## Sobre ConstraintLayout

| Pregunta                                                       | Respuesta                                                                                                                                 |
|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| ¿Qué significa <code>@dp</code> ?                              | <code>MATCH_CONSTRAINT</code> : ocupa todo el espacio definido por las restricciones. No es cero.                                         |
| ¿Por qué una vista aparece arriba a la izquierda al ejecutar?  | Le falta alguna restricción. El editor tolera restricciones incompletas; el sistema no.                                                   |
| ¿Qué es un <code>Guideline</code> ?                            | Una línea de referencia invisible, posicionada en porcentaje o en dp, que no ocupa espacio y sirve para anclar vistas.                    |
| ¿Qué es una cadena ( <i>chain</i> )?                           | Dos o más vistas restringidas mutuamente en el mismo eje. Modos: <code>spread</code> , <code>spread_inside</code> , <code>packed</code> . |
| ¿Por qué <code>Start/End</code> y no <code>Left/Right</code> ? | Porque <code>Start</code> es la derecha en idiomas RTL (árabe, hebreo). Se declara el rol, no la posición física.                         |
| ¿Qué ventaja da frente a <code>LinearLayout</code> anidado?    | Jerarquía plana: menos pasadas de medición y dibujo, mejor rendimiento y adaptabilidad.                                                   |

## Sobre estilos, temas y Material 3

| Pregunta                                                     | Respuesta                                                                                                                                                                                                                                       |
|--------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Diferencia entre estilo y tema                               | El estilo se aplica a <b>una vista</b> ( <code>style="@style/_"</code> ); el tema a <b>toda una Activity o aplicación</b> (en el manifiesto). Se escriben igual; cambia dónde se aplican.                                                       |
| ¿Cómo se hereda un estilo?                                   | Por <code>parent="..."</code> (obligatorio si el padre es externo) o por notación de punto <code>Padre.Hijo</code> (solo para estilos propios).                                                                                                 |
| ¿Qué significa el prefijo "on" en Material 3?                | Es el color del contenido que va <b>encima</b> de otro rol. <code>colorOnPrimary</code> va sobre <code>colorPrimary</code> , y el par está calculado para cumplir contraste.                                                                    |
| ¿Qué es HCT?                                                 | Espacio de color de Google: <i>Hue, Chroma, Tone</i> . El tono está calibrado contra la luminancia percibida, de modo que la distancia tonal predice el contraste.                                                                              |
| ¿Cuántas paletas genera el Theme Builder?                    | Cinco desde la semilla (primary, secondary, tertiary, neutral, neutral variant) más la de error, de matiz fijo. Cada una con trece tonos.                                                                                                       |
| ¿Qué tono es <code>colorPrimary</code> en claro y en oscuro? | T40 en claro, T80 en oscuro. <code>colorOnPrimary</code> es T100 y T20 respectivamente.                                                                                                                                                         |
| ¿Por qué falla una migración parcial a Material 3?           | Los componentes Material leen atributos del tema ( <code>shapeAppearance*</code> , <code>textAppearance*</code> ). Si el tema es AppCompat, esos atributos no existen y el componente lanza <code>IllegalArgumentException</code> en ejecución. |
| ¿Por qué nombrar por rol y no por apariencia?                | Porque el valor cambia (modo oscuro, temas dinámicos) y el nombre debe seguir siendo cierto. El rol es la interfaz del contrato entre la app y el sistema de diseño.                                                                            |
| ¿Qué exige WCAG AA para texto normal?                        | 4,5 : 1. Para texto grande y componentes de interfaz, 3 : 1.                                                                                                                                                                                    |
| ¿Cómo se activa el modo oscuro sin Java?                     | Tema con padre <code>DayNight</code> + carpeta <code>values-night</code> . El sistema cambia la bandera <code>uiMode</code> y el calificador <code>night</code> pasa a coincidir.                                                               |

## Preguntas con trampa

| Pregunta                                                                         | Respuesta correcta                                                                                                                                                 |
|----------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "¿ <code>layout-land</code> es un layout distinto?"                              | No. Es una <b>variante</b> del mismo recurso <code>R.layout.activity_main</code> . Por eso debe tener los mismos <code>id</code> .                                 |
| "¿La densidad se descarta como los demás calificadores?"                         | No. Es la única excepción: si no existe la exacta, el sistema toma la más cercana y <b>escala</b> el mapa de bits.                                                 |
| "¿ <code>sw600dp</code> cambia al rotar?"                                        | No. <code>sw</code> es el lado <b>menor</b> , que no cambia con la orientación. Por eso significa de hecho "es una tablet".                                        |
| "¿Qué tiene más precedencia: <code>land</code> o <code>sw600dp</code> ?"         | <code>sw600dp</code> . El orden va de lo estable (hardware, idioma) a lo volátil (orientación).                                                                    |
| "¿ <code>getResources().getColor()</code> sirve?"                                | Está obsoleto desde API 23 porque no recibe el tema. Lo correcto es <code>ContextCompat.getColor(context, id)</code> .                                             |
| "¿ <code>getDimension</code> y <code>getDimensionPixelSize</code> son lo mismo?" | No. La primera devuelve <code>float</code> ; la segunda devuelve <code>int</code> redondeando y garantizando al menos 1 px. Para anchos y altos se usa la segunda. |
| "Muéstrame dónde está el texto 'Bienvenido' en tu código Java."                  | No está. Está en <code>strings.xml</code> . En Java solo aparece <code>R.string.mensaje_bienvenida</code> , que es un entero.                                      |

# Tarjetas de memorización

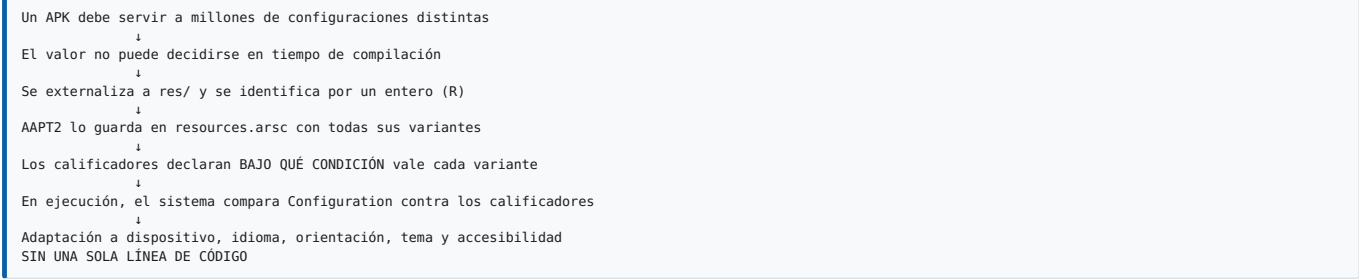
## Números que hay que saber de memoria

| Dato                                          | Valor                                                                |
|-----------------------------------------------|----------------------------------------------------------------------|
| Densidad de referencia                        | 160 dpi (mdpi)                                                       |
| Factores de densidad                          | ldpi 0,75x · mdpi 1x · hdpi 1,5x · xhdpi 2x · xxhdpi 3x · xxxhdpi 4x |
| Objetivo táctil mínimo                        | 48 dp                                                                |
| Retícula de espaciado                         | múltiplos de 8 dp                                                    |
| Contraste WCAG AA, texto normal               | 4,5 : 1                                                              |
| Contraste WCAG AA, texto grande y componentes | 3 : 1                                                                |
| minSdk / targetSdk de esta guía               | 24 / 34                                                              |
| Tonos por paleta en Material 3                | 13 (0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 95, 99, 100)              |
| Paletas generadas desde la semilla            | 5 + error                                                            |
| colorPrimary claro / oscuro                   | T40 / T80                                                            |
| Contraste del #FF5722 sobre blanco            | 3,16 : 1 (no cumple)                                                 |

## Definiciones en una línea

- **Recurso**: valor externalizado, identificado por un entero en `R` y resuelto en ejecución según la `Configuration`.
- **AAPT2**: compilador de recursos; produce `R` y `resources.arsc`.
- **Calificador**: sufijo de carpeta que produce una variante del mismo recurso.
- **dp**: unidad anclada a 160 dpi, independiente de la densidad física.
- **sp**: `dp` multiplicado además por el `fontScale` del usuario; solo para texto.
- **Estilo**: conjunto de atributos aplicado a una vista.
- **Tema**: estilo aplicado a una Activity o a toda la aplicación, consultable con `?attr/`.
- **ShapeAppearance**: definición de familia y tamaño de esquina, aplicada por categoría de componente.
- **TextAppearance**: definición de tamaño, familia, peso e interletraje, asignada a un rol tipográfico.
- **HCT**: espacio de color (matiz, croma, tono) cuyo tono está calibrado contra la luminancia percibida.
- **Rol semántico**: nombre que describe la función de un color, no su valor.

## Cadena causal que ordena todo el punto 3.7



Si en la sustentación se pierde el hilo, basta con volver a esta cadena: cualquier pregunta del punto 3.7 cae en alguno de sus eslabones.

# Lista de verificación de entrega

## Código

- ☐ Ningún literal de texto, color o dimensión en ningún archivo de `layout/`.
- ☐ `colors.xml` con cinco o más colores de nombre semántico.
- ☐ `dimens.xml` con cuatro o más dimensiones reutilizables.
- ☐ `strings.xml` con todas las cadenas visibles de la pantalla principal.
- ☐ Estilo `EstiloBotonPrincipal` definido en `themes.xml` y aplicado a ambos botones.
- ☐ Drawable `shape` con esquinas redondeadas.
- ☐ Pantalla en `ConstraintLayout` con encabezado, imagen, campo de bienvenida y dos botones.
- ☐ `layout-land/activity_main.xml` con **los mismos** `android:id`.
- ☐ `values-en/strings.xml` completo.
- ☐ `values-night/colors.xml` con los mismos nombres.
- ☐ Drawable vectorial verificado en dos densidades.
- ☐ `values-sw600dp/dimens.xml` con márgenes ampliados.
- ☐ Lint sin advertencias `HardcodedText`, `PxUsage`, `SpUsage`, `ContentDescription`.

## Repositorio

- ☐ Repositorio Git compartido, con commits **de ambos integrantes**.
- ☐ Commits con mensajes descriptivos, no “cambios” ni “fix”.
- ☐ Historial que refleje la rotación de roles.

## Informe

- ☐ Carátula con nombres completos y registro universitario de ambos.
- ☐ Tabla de roles ejercidos en cada actividad práctica.
- ☐ AT-3.7.1: mapa conceptual + tabla de recursos.
- ☐ AT-3.7.2: tabla de auditoría + tres preguntas respondidas.
- ☐ AP-3.7.1 y AP-3.7.2: fragmentos de código y capturas de ejecución.
- ☐ AC-3.7.3 (si se hizo): lámina, capturas claro/oscuro, tabla de contraste.
- ☐ Dificultades encontradas y forma de resolución.
- ☐ **Conclusiones individuales:** un párrafo propio y firmado por cada integrante.
- ☐ Anexo con `colors.xml`, `dimens.xml` y `strings.xml` impresos.

### CONDICIÓN DE ORIGINALIDAD DE LA CÁTEDRA

La guía advierte que la similitud *estructural* entre parejas es esperable, pero que la coincidencia **literal de nombres de variables, comentarios o estructuras internas** no lo es, y que el código sin trazabilidad de autoría se califica como **no presentada**.

En consecuencia: este documento es material de estudio, no un entregable. Los nombres de recursos, los comentarios y la redacción del informe deben rehacerse con criterio propio, y cada integrante debe haber tecleado y entendido lo que firma. La trazabilidad se demuestra con el historial de Git, que registra quién escribió qué y cuándo.